A Novel Quadruped Path Planner for Obstruction-Free Foothold Placement with Applications to Precision Farming

R.M. Beumer, C.R.A. Smeets, M.J.T.C. van Zutphen, and D.J. Antunes

***Abstract*— Autonomous quadrupedal robots in agricultural fields must move between densely planted crops without stepping on them. Standard foothold heuristics, however, place the feet for balance and ignore obstacles, while fixing the footholds in advance takes away the robot's ability to step for balance. This paper formulates footstep planning as a robust problem in which the footholds generated by a Raibert/capture-point law must remain collision-free after the balance corrections the robot applies during execution. To solve it, we introduce the Best-first Online Backtracking State Tree Planner (BOBSTP), which searches over body-velocity trajectories by combining best-first expansion with adaptive stochastic exploration and pruning with return to an ancestor, and which checks footholds, swept regions and adaptive Bézier swing trajectories against an inflated obstacle set. For a step-to-step model of the balance error under the closed-loop stance controller, identified from data, we derive the inflation margin that guarantees collision-free footholds and bounded balance error under bounded disturbances, and show when footholds fixed by the plan suffice. Simulations with the MIT Mini Cheetah in randomly cluttered fields show average planning times below 0.125 seconds per planned meter and stable tracking of the planned paths, and trace the remaining obstacle contacts to swing tracking errors beyond the assumed bound.**

I. Introduction

When performing weeding, inspecting, or other management tasks in agricultural fields, autonomous robots must navigate between plants, through narrow passageways, and over uneven soil while minimizing crop damage. Wheeled and tracked robots require destructive clearance, whereas legged robots could leverage deliberate leg placement to minimize crop disturbance while maintaining mobility. Quadrupeds have recently been proposed and tested in agriculture for mechanical weeding [1] and for semantic mapping and inspection of crops [2]. However, most quadruped controllers rely on predefined gaits and heuristic foothold placement, leaving obstacle avoidance to disturbance rejection by the whole-body controller. Planning the footholds around crops, moreover, conflicts with their role in balance: a robot whose footholds are fixed in advance can no longer correct its balance by stepping.

This paper addresses both issues with an online footstep planner for local navigation. The obstacles are assumed known; perception, global path optimization and whole-body control are outside the scope, but the planner is designed to integrate with existing higher-level navigation. The main contributions are:

- The *Best-first Online Backtracking State Tree Planner (BOBSTP)*, which generates a body-velocity trajectory together with a collision-free foothold sequence. It combines best-first expansion with adaptive stochastic exploration and pruning with return to an ancestor, and checks each step, i.e., the footholds given by a Raibert/capture-point law, the region swept by the leg and an adaptive Bézier swing whose clearance adjusts to the local obstacle heights, against an inflated obstacle set.
- A *robust foothold margin* that certifies the planned footholds under balance corrections. For a step-to-step model of the divergent-component-of-motion error under the closed-loop stance controller, identified from data, we derive the obstacle inflation that guarantees collision-free footholds and swings and a bounded balance error under bounded disturbances. We show when footholds fixed by the plan suffice, and how the feedback gain trades the balance error against the margin.
- A simulation study with the MIT Mini Cheetah in randomly cluttered fields with up to 70 obstacles per square meter, showing average planning times below 0.125 s per planned meter and stable tracking of the planned paths.



Fig. 1. Boston Dynamics' Spot in an agricultural setting.

Quadruped locomotion requires coordinated gait timing, foot motion, and contact forces [3]. On the Mini Cheetah, the stance forces are computed by convex MPC and realized by whole-body control [4]–[6], while the footholds are chosen heuristically from the body motion: the Raibert heuristic

The authors are with the Department of Mechanical Engineering, Eindhoven University of Technology, Eindhoven, The Netherlands.

This research was conducted as part of the Synergia consortium. The Synergia project is organized and led by Wageningen University and Research in close cooperation with Next Food Collective as well as the Universities of Delft, Twente, Eindhoven, and Nijmegen. The authors have declared that no competing interests exist in the writing of this publication. Funding for this research was obtained from the Netherlands Organisation for Scientific Research (NWO grant 17626), IMEC-One Planet and other private parties.

combines a prediction of body travel over half the stance duration with feedback on the velocity error [7], and capture-point methods place the foot where a linear inverted pendulum comes to rest [8], [9]. Foot placement is thus a main balance mechanism of a walking robot, yet these heuristics ignore obstacles, and terrain-aware extensions mainly steer the feet away from unsuitable ground, such as slippery regions [10], [11], rather than away from objects that must not be touched.

Avoiding such objects calls for planning the footholds explicitly. Search-based planners evaluate sequences of candidate footholds, using best-first search for quadrupeds under kinematic and static-balance constraints but without timing or dynamics [12], and A* variants for bipeds [13]–[15], whereas optimization-based planners encode contact feasibility as mixed-integer constraints [16]. Clearance during the swing is usually left to fixed Bézier or spline profiles [17], [18], which do not account for intervening obstacles. Closest to our work, Risbourg et al. [19] combine contact selection by a mixed-integer program with collision-aware Bézier swings computed by quadratic programs; however, collisions are checked only against the contact surfaces of the step, and adapting the body trajectory when no feasible contact plan exists is left as future work.

Moreover, once the footholds are fixed by a planner, they can no longer serve for balance, and, to the best of our knowledge, no existing method quantifies how far a planned foothold may be moved by balance corrections without touching an obstacle. The present paper addresses both issues. BOBSTP searches over body-motion trajectories, so the body motion adapts when no feasible foothold sequence exists, and it checks footholds, swept regions and adaptive swings against obstacles inflated by a margin that we prove remains valid under balance corrections. More related work can be found in [20].

The remainder of this paper is organized as follows. Section II formulates the problem, Section III presents BOBSTP, including the adaptive swing trajectory, Section IV derives the robust foothold margin, Section V reports the simulation results, and Section VI concludes the paper. The proofs are given in Appendix I.

II. Problem Formulation

A. Body model

Consider a quadrupedal robot with a body-fixed frame R located at its center of mass (CoM), operating in the horizontal (x, y)-plane of the world frame W. Motion outside this plane is neglected and the CoM height h is assumed constant. Unless stated otherwise, positions are expressed in W; $\mathbf{p} \in \mathbb{R}^2$ denotes a planar position and $\bar{\mathbf{p}} \in \mathbb{R}^3$ its 3D counterpart.

The state of the body consists of its planar pose $\boldsymbol{\xi} = [\,x_R \;\; y_R \;\; \theta_R\,]^\top$, i.e., the position and yaw of R in W, and its planar twist $\mathbf{v} = [\,u \;\; v \;\; \omega\,]^\top$, expressed in R, so that

$$\mathbf{q} := \begin{bmatrix} \boldsymbol{\xi} \\ \mathbf{v} \end{bmatrix} \in \mathbb{R}^6, \qquad \dot{\boldsymbol{\xi}} = \mathbf{X}(\theta_R)\,\mathbf{v}, \tag{1}$$

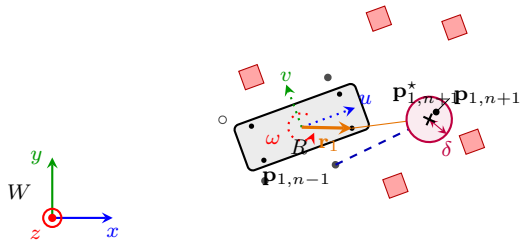


Fig. 2. Top view of the robot with the world frame W (solid axes) and the body frame R (dotted), whose planar twist has components u, v, ω. For the front-right leg, the hip offset $\mathbf{r}_1$ (orange) and the swing (dashed) from the lift-off foothold $\mathbf{p}_{1,n-1}$ to the nominal foothold $\mathbf{p}^\star_{1,n+1}$ of (5), placed ahead of the hip by the Raibert offset. Balance corrections move the realized foothold $\mathbf{p}_{1,n+1}$ within a disc of radius δ, which the planner keeps free of obstacles (red).

with

$$\mathbf{X}(\theta) := \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{2}$$

We write $\mathbf{p}_R := [\,x_R \;\; y_R\,]^\top$ for the CoM position and $\dot{\mathbf{p}}_R \in \mathbb{R}^2$ for its world-frame velocity, i.e., the first two components of $\dot{\boldsymbol{\xi}}$. The control input is the desired body acceleration $\mathbf{a} = \dot{\mathbf{v}}_{des}$, which lower-level controllers track. The planned velocities $\mathbf{v}$ are passed to the locomotion controller as references. The realized velocity and acceleration satisfy the element-wise bounds

$$\mathbf{v}_{\min} \le \mathbf{v} \le \mathbf{v}_{\max}, \qquad \dot{\mathbf{v}}_{\min} \le \dot{\mathbf{v}} \le \dot{\mathbf{v}}_{\max}. \tag{3}$$

B. Gait and foothold law

The legs are indexed by $i \in \{1, 2, 3, 4\}$ (front right, front left, rear right, rear left). The robot trots: the diagonal pairs $\mathcal{P}_A = \{1, 4\}$ and $\mathcal{P}_B = \{2, 3\}$ alternate in stance, each for a stance duration T_s, with swing and stance of equal duration. Let $n \in \mathbb{N}$ index the steps, i.e., the touchdown instants $t_n := t_0 + n\,T_s$, and let $\mathcal{S}_n \in \{\mathcal{P}_A, \mathcal{P}_B\}$ be the pair in stance on $[t_n, t_{n+1})$, with $\mathcal{S}_{n+1} \neq \mathcal{S}_n$. The feet $i \in \mathcal{S}_{n+1}$ lift off at t_n, swing on $[t_n, t_{n+1})$ from the foothold $\mathbf{p}_{i,n-1}$, where they stood since $\mathcal{S}_{n-1} = \mathcal{S}_{n+1}$, and land at the foothold $\mathbf{p}_{i,n+1} \in \mathbb{R}^2$ (on flat ground, $\bar{\mathbf{p}}_{i,n+1} = [\,\mathbf{p}_{i,n+1}^\top \;\; 0\,]^\top$).

Footholds are generated by the standard Raibert/capture-point law used on the Mini Cheetah [4], [7], [21], which we split into a part that depends only on the *planned* motion and a balance feedback. Let $(\cdot)^\star$ denote planned quantities. For $i \in \mathcal{S}_{n+1}$,

$$\mathbf{p}_{i,n+1} = \mathbf{p}^\star_{i,n+1} + \Delta\mathbf{p}_{n+1}, \tag{4}$$

where the nominal foothold is

$$\begin{aligned} \mathbf{p}^\star_{i,n+1} := \mathbf{p}^\star_R(t_{n+1}) + \mathbf{R}(\theta^\star(t_{n+1}))\,\mathbf{r}_i + \tfrac{1}{2}T_s\,\dot{\mathbf{p}}^\star_R(t_n) \\ - \tfrac{h}{2g}\,\omega^\star(t_n)\,\vec{z}_W \times \dot{\mathbf{p}}^\star_R(t_n), \end{aligned} \tag{5}$$

with $\mathbf{R}(\theta) \in \mathrm{SO}(2)$ the planar rotation, $\mathbf{r}_i$ the constant offset from the CoM to the hip of leg i, expressed in R (Fig. 2), and $\vec{z}_W$ the vertical unit vector, so that $\vec{z}_W \times \dot{\mathbf{p}}^\star_R$ is the planar velocity rotated by $90°$. The first two terms are the hip location at touchdown, the third is Raibert's neutral point,

and the last compensates the centripetal acceleration when turning. The correction $\Delta\mathbf{p}_{n+1}$ is common to both feet of the pair and is discussed below.

C. Swing trajectory

The swing of foot $i \in \mathcal{S}_{n+1}$ is generated by a map Γ from the lift-off foothold $\mathbf{p}_{i,n-1}$, the touchdown foothold $\mathbf{p}_{i,n+1}$ and the obstacle set to a curve $\bar{\mathbf{p}}_{\text{swing},i}(t)$, $t \in [t_n, t_{n+1}]$. With phase $\phi = (t - t_n)/T_s \in [0,1]$ and the cubic blending function $B(\phi) = 3\phi^2 - 2\phi^3$, its horizontal part is

$$\mathbf{p}_{\text{swing},i}(\phi) = \big(1 - B(\phi)\big)\,\mathbf{p}_{i,n-1} + B(\phi)\,\mathbf{p}_{i,n+1}, \quad (6)$$

while its height $z_{\text{swing},i}(\phi)$ is shaped by the adaptive Bézier construction of Section III-E so as to clear the obstacles encountered along the segment. Since $B(\phi) \in [0,1]$, (6) is a convex combination of the two footholds, a property used in Section IV.

D. Balance model

To analyse balance we approximate the horizontal CoM dynamics by the linear inverted pendulum (LIP)

$$\ddot{\mathbf{p}}_R = \Omega^2(\mathbf{p}_R - \mathbf{c}), \qquad \Omega := \sqrt{g/h}, \quad (7)$$

where $\mathbf{c} \in \mathbb{R}^2$ is the center of pressure (CoP). For point contacts on flat ground with ground reaction forces $\mathbf{f}_i = [\,f_i^x \quad f_i^y \quad f_i^z\,]^\top$ at the stance feet, it is the normal-force-weighted average $\mathbf{c} = \sum_{i\in\mathcal{S}_n} f_i^z\,\mathbf{p}_{i,n} \big/ \sum_{i\in\mathcal{S}_n} f_i^z$; since $f_i^z \geq 0$, it lies in the convex hull of the stance feet, and the stance-force controller moves it by redistributing the f_i^z. The model (7) follows from Newton's law for the CoM under constant height, $\sum_i f_i^z = mg$, with m the robot mass, and constant centroidal angular momentum. Its unstable part is captured by the divergent component of motion (DCM), or capture point [8], [9],

$$\boldsymbol{\zeta} := \mathbf{p}_R + \tfrac{1}{\Omega}\dot{\mathbf{p}}_R. \quad (8)$$

Differentiating (8) and using (7) gives $\dot{\boldsymbol{\zeta}} = \Omega(\boldsymbol{\zeta} - \mathbf{c})$, while $\dot{\mathbf{p}}_R = \Omega(\boldsymbol{\zeta} - \mathbf{p}_R)$, so the CoM converges to the DCM. The DCM is thus repelled by the CoP: the robot comes to rest only if the CoP is placed on the DCM, and it can keep the DCM bounded only by moving the CoP, i.e., by redistributing forces among the stance feet or by stepping. For the trot, this hull is the segment joining the two stance feet; we write $\mathbf{c}(t) = \bar{\mathbf{c}}_n + \boldsymbol{\nu}(t)$, with the virtual foot $\bar{\mathbf{c}}_n := \frac{1}{2}\sum_{i\in\mathcal{S}_n} \mathbf{p}_{i,n}$. Solving $\dot{\boldsymbol{\zeta}} = \Omega(\boldsymbol{\zeta} - \mathbf{c})$ over $[t_n, t_{n+1}]$ gives the step-to-step map

$$\boldsymbol{\zeta}_{n+1} = a\,(\boldsymbol{\zeta}_n - \bar{\mathbf{c}}_n) + \bar{\mathbf{c}}_n + \mathbf{w}_n, \qquad a := e^{\Omega T_s} > 1, \quad (9)$$

where $\boldsymbol{\zeta}_n := \boldsymbol{\zeta}(t_n)$ and

$$\mathbf{w}_n = -\Omega \int_{t_n}^{t_{n+1}} e^{\Omega(t_{n+1}-s)}\,\boldsymbol{\nu}(s)\,ds \quad (10)$$

is the effect of the CoP modulation $\boldsymbol{\nu}$ by the stance-force controller. Since $a > 1$, (9) is unstable when the CoP is held at the virtual foot.

Let $\boldsymbol{\zeta}_n^\star$ be the DCM of the planned motion and $\mathbf{e}_n := \boldsymbol{\zeta}_n - \boldsymbol{\zeta}_n^\star$. Both feet of the pair are shifted by the same $\Delta\mathbf{p}_n$, so the virtual foot is $\bar{\mathbf{c}}_n = \bar{\mathbf{c}}_n^\star + \Delta\mathbf{p}_n$. If the CoP stayed at the virtual foot, subtracting the nominal version of (9) would give $\mathbf{e}_{n+1} = a(\mathbf{e}_n - \Delta\mathbf{p}_n) + \Delta\mathbf{p}_n + \mathbf{w}_n$. On a real robot, however, the stance controller does not hold the CoP there: it tracks the planned motion by redistributing the contact forces and by using the body's angular momentum, so it acts on $\mathbf{e}_n$ itself and partly cancels the effect of $\Delta\mathbf{p}_n$. Moreover, the CoP can move only along the stance diagonal, which alternates between $\mathcal{P}_A$ and $\mathcal{P}_B$, so the response differs between the two horizontal directions and between the two pairs. We therefore describe the error by the closed-loop stance map

$$\mathbf{e}_{n+1} = \boldsymbol{\Sigma}_n\,\mathbf{e}_n + \boldsymbol{\Lambda}_n\,\Delta\mathbf{p}_n + \mathbf{w}_n, \quad (11)$$

where the matrices $\boldsymbol{\Sigma}_n, \boldsymbol{\Lambda}_n \in \mathbb{R}^{2\times 2}$ capture the combined effect of the pendulum dynamics and the stance controller during step n, and $\mathbf{w}_n$ collects the remaining tracking errors and model mismatch, with $\|\mathbf{w}_n\| \leq \bar{w}$. Expressed in the body frame, $\boldsymbol{\Sigma}_n$ and $\boldsymbol{\Lambda}_n$ depend only on the stance pair $\mathcal{S}_n$, so for steady walking they alternate between two pairs of matrices. The LIP with the CoP fixed at the virtual foot is the special case $\boldsymbol{\Sigma}_n = a\mathbf{I}$, $\boldsymbol{\Lambda}_n = (1-a)\mathbf{I}$. In general, the matrices depend on the robot, its stance controller, the gait and the speed, and are identified from data, for instance by least squares on recorded steps whose footholds are perturbed independently of the error (Section V).

The balance feedback in (4) is the correction based on the DCM error predicted at touchdown with (11),

$$\Delta\mathbf{p}_{n+1} = \mathbf{K}_{n+1}\,\hat{\mathbf{e}}_{n+1}, \qquad \hat{\mathbf{e}}_{n+1} := \boldsymbol{\Sigma}_n\,\mathbf{e}_n + \boldsymbol{\Lambda}_n\,\Delta\mathbf{p}_n, \quad (12)$$

computed at lift-off t_n from measured quantities, with gain matrix $\mathbf{K}_{n+1} \in \mathbb{R}^{2\times 2}$, which may likewise depend on the pair. For the LIP values and $\mathbf{K}_n = K\mathbf{I}$ this is the capture-point correction. The case $\mathbf{K}_n = \mathbf{0}$ corresponds to footholds fixed by the plan, and the original Raibert law uses only the velocity part of $\mathbf{e}_n$.

E. Problem statement

Let $\mathcal{O} \subset \mathbb{R}^3$ be the set of static obstacles (e.g., plants), and let $\mathcal{R}_i(n)$ be the region swept by the shoulder and body of leg i during step n, defined from $\mathbf{p}_{i,n-1}$ and $\mathbf{p}_{i,n+1}$ as in (24).

Problem 1. Given $\mathcal{O}$, an initial state $\mathbf{q}_0$, a goal $\mathbf{p}_G$ with tolerance $r > 0$, the bounds (3), a stance map $(\boldsymbol{\Sigma}_n, \boldsymbol{\Lambda}_n)$ with disturbance bound $\bar{w}$, and gains $\mathbf{K}_n$, find a planned body trajectory $\mathbf{q}^\star(t)$, $t \in [0, T]$, and hence nominal footholds (5), such that

1) $\mathbf{q}^\star$ satisfies (3) and $\|\mathbf{p}_R^\star(T) - \mathbf{p}_G\| \leq r$;
2) for every disturbance sequence with $\|\mathbf{w}_n\| \leq \bar{w}$, the realized footholds (4), swing trajectories and swept regions avoid the obstacles, i.e., for all n and $i \in \mathcal{S}_{n+1}$,

$$\begin{aligned} &\bar{\mathbf{p}}_{i,n+1} \notin \mathcal{O}, \qquad \mathcal{R}_i(n) \cap \mathcal{O} = \emptyset, \\ &\bar{\mathbf{p}}_{\text{swing},i}(t) \notin \mathcal{O} \quad \forall\, t \in [t_n, t_{n+1}]; \end{aligned} \quad (13)$$

3) the DCM error remains bounded, $\sup_n \|\mathbf{e}_n\| < \infty$.

Algorithm 1. Pseudocode of the BOBSTP algorithm.

```
def BOBSTP(root):
  tree = init_tree(root)
  cur_node = tree.root
  while root.n_samples < N_max,samples:
    if cur_node.n_samples >= N_max,samples:
      tree.prune(cur_node)
      cur_node = cur_node.ancestor(N_ancestor)
      continue
    a_nom = nominal_input(cur_node.q)  # (18)
    a = sample_input(a_nom, cur_node.n_samples)
    potential_next_node = step(cur_node.q, a)
    footholds = calc_footholds(cur_node.q, a)
    if is_feasible(cur_node, a, footholds): # (23)
      cur_node.add_child(potential_next_node,
                         footholds)
      cur_node.n_success += 1
      cur_node = potential_next_node
    else:
      cur_node.n_fail += 1
    if goal_reached(cur_node):
      path = path_from_tree(tree, cur_node)
      return path, [node.footholds
                    for node in path]
```

Requirement 3 is what keeps the robot upright; requirement 2 asks that the footholds remain safe *after* the balance correction. Planning with $\bar{w} = 0$ and $\Delta\mathbf{p} \equiv 0$, enforces 1 and 2 only; Section IV shows that it then satisfies 3 if the stance controller alone stabilizes the error, and fails to if the stance controller alone is unstable. BOBSTP (Section III) solves Problem 1 by searching over $\mathbf{q}^\star$ while checking (13) on an inflated obstacle set, and Section IV gives the inflation that makes this check sufficient.

III. The BOBSTP Planner

The *Best-first Online Backtracking State Tree Planner* (BOBSTP) is a novel hybrid motion planning algorithm for the incremental exploration of complex state spaces. Each node in the tree corresponds to a *feasible* robot state (Section III-A), i.e., a state that satisfies the state constraints (3) and the obstacle checks of Section III-D, and each edge represents a valid control input that transitions the system from one state to another without violating any system constraints. To progress rapidly toward the goal, the planner greedily selects the most promising control input at each node (Section III-B). When this option is infeasible, for example due to foot placement, alternative inputs are sampled stochastically with an increasing exploration range. When a node reaches its sampling limit, pruning removes its subtree and moves the active search to an ancestor, where sampling resumes. The algorithm is summarized in Algorithm 1 and illustrated in Fig. 3; its elements are detailed in the following subsections.

A. Discrete time state model

For sample-based planning, the state trajectory $\mathbf{q}(t)$ is discretized with a fixed time step $\Delta t > 0$. Each node in the tree represents the state at time $t = k\Delta t$, $k \in \mathbb{N}$, denoted

$$\mathbf{q}_k := \begin{bmatrix} \boldsymbol{\xi}_k \\ \mathbf{v}_k \end{bmatrix}, \tag{14}$$

following (1). Each edge represents the transition from a parent node to its child under a control input $\mathbf{a}_{k+1}$. Discretizing (1) with the semi-implicit Euler scheme, with the rotation $\mathbf{X}(\theta_k)$ of (2) evaluated at the start of the interval, gives

$$\mathbf{q}_{k+1} = \begin{bmatrix} \boldsymbol{\xi}_{k+1} \\ \mathbf{v}_{k+1} \end{bmatrix} = \begin{bmatrix} \boldsymbol{\xi}_k + \Delta t \mathbf{X}(\theta_k)\,(\mathbf{v}_k + \Delta t \mathbf{a}_{k+1}) \\ \mathbf{v}_k + \Delta t \mathbf{a}_{k+1} \end{bmatrix}, \tag{15}$$

i.e., the velocity is updated first and the position is then propagated with the new velocity $\mathbf{v}_{k+1}$, the velocity commanded over the interval.

B. Control input sampling

For clarity of analysis, a virtual nonholonomic constraint, $v = 0$, restricts lateral motion in the model; it is not essential to the algorithm and can be relaxed without affecting the validity of the approach.

Nodes are expanded by an iterative sample-and-check process. Successor samples are drawn from a uniform distribution around a nominal input $\mathbf{a}_{k+1,\text{nom}}$, whose width is zero for the first sample, which is thus the nominal input itself, and grows with the number of samples. Each sample is checked against (3) and the transition checks of Section III-D and, if feasible, added to the tree.

The nominal input $\mathbf{a}_{k+1,\text{nom}}$ is the acceleration that drives the body velocity toward a *desired velocity* $\mathbf{v}_{k+1,\text{des}}$ pointing to the goal. The desired forward and angular velocities are computed from the error between the planar pose $\boldsymbol{\xi}_k$ and the goal position $\mathbf{p}_G = [\,x_G \;\; y_G\,]^\top$:

$$\begin{aligned} u_{k+1,\text{des}} &= \max\{0,\ \frac{\alpha_k}{\Delta t}(\cos(\theta_k)\,(x_G - x_k) \\ &\qquad + \sin(\theta_k)\,(y_G - y_k))\}, \\ \omega_{k+1,\text{des}} &= \frac{\tilde{\theta}_{e,k}}{\Delta t}, \\ \tilde{\theta}_{e,k} &:= \text{wrap}_{(-\pi,\pi]}(\text{atan2}\,(y_G - y_k,\ x_G - x_k) - \theta_k), \end{aligned} \tag{16}$$

where the yaw error $\tilde{\theta}_{e,k}$ is wrapped to $(-\pi, \pi]$. The heuristic factor $\alpha_k = \left|\cos(\tilde{\theta}_{e,k})^p\right|$ attenuates the forward velocity when the robot is misaligned with the goal, which mitigates spiraling; it equals 1 when the robot faces the goal and decreases to 0 at a misalignment of $\pm\pi/2$, more sharply for larger p. With the nonholonomic constraint and the limits (3), the desired velocity vector is

$$\mathbf{v}_{k+1,\text{des}} = \begin{bmatrix} \min\{u_{\max}, \max\{u_{\min}, u_{k+1,\text{des}}\}\} \\ 0 \\ \min\{\omega_{\max}, \max\{\omega_{\min}, \omega_{k+1,\text{des}}\}\} \end{bmatrix}, \tag{17}$$

and the nominal input is the finite-difference acceleration toward it, saturated to (3):

$$\mathbf{a}_{k+1,\text{nom}} = \min\{\dot{\mathbf{v}}_{\max}, \max\{\dot{\mathbf{v}}_{\min}, \frac{\mathbf{v}_{k+1,\text{des}} - \mathbf{v}_k}{\Delta t}\}\}. \tag{18}$$

Using LaSalle's invariance principle, Appendix II shows that this input, applied to the model (15), drives the state to the goal position and orientation.

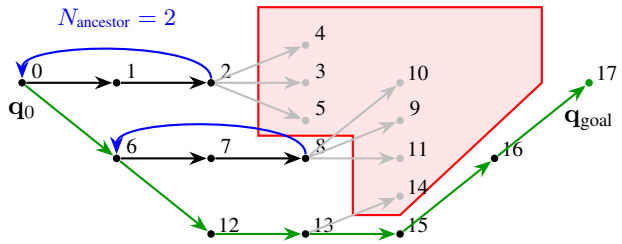


Fig. 3. Search tree with pruning ($N_{\text{max,samples}} = 3$, $N_{\text{ancestor}} = 2$). Nodes are states $\mathbf{q}_k$ and edges accelerations $\mathbf{a}_{k+1}$; only the x- and y-components are drawn. Numbers give the order in which samples were drawn; the red area is an obstruction region. Black: active branches; gray: rejected samples; blue arrows: return to an ancestor.

C. Pruning and Stochastic Sampling

a) Pruning with ancestor return: Each node i counts its succession attempts $n_{\text{samples},i} = n_{\text{success},i} + n_{\text{fail},i}$, the sum of its successfully generated children and its failed attempts, i.e., samples rejected by the checks of Section III-D.

Pruning is triggered when $n_{\text{samples},i} = N_{\text{max,samples}}$. At that point, all descendants of node i are removed from the tree, discarding the unpromising branch; node i is marked as exhausted, and the active search resumes at its N_{ancestor}-th ancestor. N_{ancestor} thus sets how far up the tree the search backtracks (Algorithm 1, Fig. 3). This is also how $n_{\text{success},i'}$ of an ancestor i' can exceed 1. If all children of a node keep failing or being pruned, the node itself eventually reaches $n_{\text{samples},i} = N_{\text{max,samples}}$ and is pruned in turn, which prevents over-committing to unproductive regions.

b) Stochastic Sampling: As $n_{\text{samples},i}$ increases, the sampling window around $\mathbf{a}_{k+1,\text{nom}}$ grows, so the planner deviates further from the nominal input when samples keep failing, which increases the likelihood of finding a feasible path. The limits of the uniform sampling set $\mathcal{U} := [\mathbf{a}^-, \mathbf{a}^+]$ for $\mathbf{a}_{k+1}$ scale with the exploration magnitude $\mu_{\text{explore}} \in [0, 1]$ as

$$\begin{aligned} \mathbf{a}^- &= \mathbf{a}_{k+1,\text{nom}} - \mu_{\text{explore}} \left(\mathbf{a}_{k+1,\text{nom}} - \hat{\dot{\mathbf{v}}}_{\min,k} \right), \\ \mathbf{a}^+ &= \mathbf{a}_{k+1,\text{nom}} + \mu_{\text{explore}} \left(\hat{\dot{\mathbf{v}}}_{\max,k} - \mathbf{a}_{k+1,\text{nom}} \right), \end{aligned} \tag{19}$$

where $\hat{\dot{\mathbf{v}}}_{\min,k} = \max\{\dot{\mathbf{v}}_{\min}, (\mathbf{v}_{\min} - \mathbf{v}_k)/\Delta t\}$ and $\hat{\dot{\mathbf{v}}}_{\max,k} = \min\{\dot{\mathbf{v}}_{\max}, (\mathbf{v}_{\max} - \mathbf{v}_k)/\Delta t\}$. The exploration magnitude grows with $n_{\text{samples},i}$ as

$$\mu_{\text{explore}} = 1 - \left(\frac{N_{\text{max,samples}} - n_{\text{samples},i}}{N_{\text{max,samples}}} \right)^{\eta}, \tag{20}$$

where the exploration factor $\eta \geq 0$ sets how quickly the sampling window grows. For $\eta = 0$, only $\mathbf{a}_{k+1,\text{nom}}$ is sampled. For any $\eta > 0$, the window approaches the full permissible range $[\hat{\dot{\mathbf{v}}}_{\min,k}, \hat{\dot{\mathbf{v}}}_{\max,k}]$ as $n_{\text{samples},i}$ approaches $N_{\text{max,samples}}$, but a larger η widens it more within the first samples.

D. Transition feasibility

Between nodes, the planned motion is reconstructed by linear interpolation,

$$\mathbf{q}^\star(t) = \mathbf{q}_k + \tfrac{t - k\Delta t}{\Delta t} \big(\mathbf{q}_{k+1} - \mathbf{q}_k \big), \quad t \in (k\Delta t, (k{+}1)\Delta t], \tag{21}$$

which defines the nominal footholds (5) and the planned DCM $\boldsymbol{\zeta}_n^\star$ at the step times. A piecewise-constant reconstruction $\mathbf{q}^\star(t) = \mathbf{q}_{k+1}$ would make this reference jump at the node times; since T_s and Δt are in general incommensurate, the jumps enter the residual $\mathbf{w}_n$ as a periodic reference defect that grows with speed (Section V). A step n is *completed* in the interval of node $k+1$ when its touchdown $t_{n+1} \in (k\Delta t, (k{+}1)\Delta t]$. At that point all planned quantities entering the nominal footholds (5) of the pair $\mathcal{S}_{n+1}$ are known, so the step can be checked; with $T_s < \Delta t$, several steps may be completed in one interval, and with $T_s > \Delta t$ possibly none.

The checks are performed against the inflated obstacle set

$$\mathcal{O}_\delta := \mathcal{O} \oplus \big(\mathcal{B}_\delta^2 \times \{0\} \big) \oplus \mathcal{B}_\varepsilon^3, \tag{22}$$

where $\oplus$ denotes the Minkowski sum, $\mathcal{B}_r^d$ the closed ball of radius r in $\mathbb{R}^d$, $\delta \geq 0$ the horizontal foothold margin and $\varepsilon \geq 0$ a bound on the swing-foot tracking error. Section IV specifies δ; with $\delta = \varepsilon = 0$, $\mathcal{O}_\delta = \mathcal{O}$. For step n and $i \in \mathcal{S}_{n+1}$, the planner requires

$$\bar{\mathbf{p}}_{i,n+1}^\star \notin \mathcal{O}_\delta, \tag{23a}$$

$$\mathcal{R}_i^\star(n) \cap \mathcal{O}_\delta = \emptyset, \tag{23b}$$

$$\bar{\mathbf{p}}_{\text{swing},i}^\star(\phi) \notin \mathcal{O}_\delta \quad \forall\, \phi \in [0, 1]. \tag{23c}$$

Here $\bar{\mathbf{p}}_{\text{swing},i}^\star$ is the swing (6) between the nominal footholds $\mathbf{p}_{i,n-1}^\star$ and $\mathbf{p}_{i,n+1}^\star$, with the height profile of Section III-E computed from $\mathcal{O}_\delta$, and

$$\mathcal{R}_i^\star(n) := \Big\{ \big[(1-s)\, \mathbf{p}_{i,n+1}^\star + s\, \mathbf{p}_{i,n-1}^\star \big] \times \{z\} \;\Big|\; s \in [0,1],\ z \in [h_{\text{step}}, h_{\text{robot}}] \Big\} \tag{24}$$

is the vertical strip between the two footholds that represents the shoulder and body of the leg, from the maximum step height h_{step} up to the robot height h_{robot} (Fig. 4). A candidate node is added to the tree if it satisfies (3) and (23) holds for every step completed in its interval and both feet of the corresponding pair; otherwise the sample counts as a failure in $n_{\text{fail},i}$.

E. Adaptive swing height

The horizontal swing is fixed by (6); only its height $z(\phi)$ adapts to the obstacles. On flat ground, the standard profile rises to the step height h_{step} at $\phi = \frac{1}{2}$ and descends again, each half being the cubic blend B. Its rise is too slow to clear an obstacle close to the lift-off point.

The adaptive profile is composed of a rise, a plateau and a descent,

$$z(\phi) = \begin{cases} h_{\text{step}}\, B(\phi/\phi_1), & \phi \in [0, \phi_1], \\ h_{\text{step}}, & \phi \in (\phi_1, \phi_2), \\ h_{\text{step}}\, B\big((1-\phi)/(1-\phi_2)\big), & \phi \in [\phi_2, 1], \end{cases} \tag{25}$$

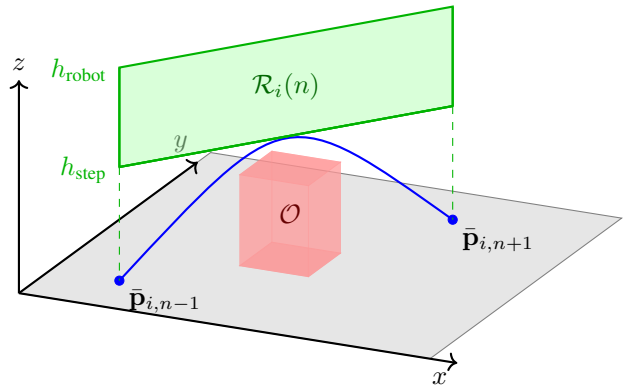


Fig. 4. Footstep transition from $\bar{\mathbf{p}}_{i,n-1}$ to $\bar{\mathbf{p}}_{i,n+1}$ with the swing trajectory (blue). The green rectangle is the swept region $\mathcal{R}_i(n)$ between the footholds, from height h_{step} to h_{robot}; the red polyhedron illustrates an obstacle.

with $0 < \phi_1 \le \frac{1}{2} \le \phi_2 < 1$. Since $B(0) = 0$, $B(1) = 1$ and $B'(0) = B'(1) = 0$, the profile is continuously differentiable for any such ϕ_1, ϕ_2. To choose them, the segment is sampled at $\phi_j = j/N_{\text{obs}}$, $j = 1, \dots, N_{\text{obs}} - 1$, and o_j denotes the height of $\mathcal{O}_\delta$ above the horizontal swing point $\mathbf{p}_{\text{swing},i}(\phi_j)$ (zero if there is none). If $\max_j o_j \ge h_{\text{step}}$, the obstacle cannot be stepped over and the transition is rejected. Otherwise, let $\beta_j := B^{-1}(o_j/h_{\text{step}}) \in [0, 1)$, i.e., the root in $[0, 1]$ of $3\beta^2 - 2\beta^3 = o_j/h_{\text{step}}$. Because B is increasing on $[0, 1]$, the rise clears sample j if and only if $\phi_j/\phi_1 \ge \beta_j$, and the descent if and only if $(1 - \phi_j)/(1 - \phi_2) \ge \beta_j$. Hence the largest admissible rise and descent durations are

$$\begin{aligned} \phi_1 &= \min\Big\{\tfrac{1}{2},\ \min_{j:\, o_j > 0} \frac{\phi_j}{\beta_j}\Big\}, \\ 1 - \phi_2 &= \min\Big\{\tfrac{1}{2},\ \min_{j:\, o_j > 0} \frac{1 - \phi_j}{\beta_j}\Big\}, \end{aligned} \tag{26}$$

and on the plateau every sample is cleared since $o_j < h_{\text{step}}$. The profile therefore clears every sampled obstacle point, and it reduces to the standard one when no obstacle is present. With $\mathcal{O}$ known and the sampling fine relative to the obstacle size, this is how (23c) is evaluated.

IV. Robust Foothold Margin

This section shows how large the margin δ in (22) must be for the plan returned by BOBSTP to solve Problem 1, and when footholds fixed by the plan suffice; Fig. 2 illustrates the margin. All proofs are given in Appendix I. Norms of matrices are the induced Euclidean norms.

Assumption 1. *(i) The DCM error obeys* (11) *with* $\|\mathbf{w}_n\| \le \bar{w}$ *and the correction* (12)*, and the robot starts on the plan,* $\mathbf{e}_0 = \mathbf{0}$*,* $\Delta\mathbf{p}_0 = \mathbf{0}$*. (ii) The executed swing of foot* $i \in \mathcal{S}_{n+1}$ *has the horizontal path* (6) *between the realized footholds* $\mathbf{p}_{i,n-1}$ *and* $\mathbf{p}_{i,n+1}$ *and the height profile computed at planning time, and the foot tracks this reference within* ε*.*

Part (ii) states that the correction shifts the swing horizontally but does not change its height profile, which is decided by the planner on $\mathcal{O}_\delta$. The tracking bound ε is a property of the swing-leg controller; the guarantee below holds for whatever value of ε that controller achieves.

Let $\mathbf{M}_n := \mathbf{\Sigma}_n + \mathbf{\Lambda}_n\mathbf{K}_n$ be the closed-loop matrix of step n and $\mathbf{\Phi}(k, m) := \mathbf{M}_k\mathbf{M}_{k-1}\cdots\mathbf{M}_{m+1}$ for $k > m$, with $\mathbf{\Phi}(m, m) := \mathbf{I}$.

Lemma 1. *Under Assumption 1(i), for* $n \ge 1$*,*

$$\mathbf{e}_n = \sum_{j=0}^{n-1} \mathbf{H}_{n,j}\,\mathbf{w}_j, \qquad \Delta\mathbf{p}_n = \mathbf{K}_n \sum_{j=0}^{n-2} \mathbf{H}_{n,j}\,\mathbf{w}_j, \tag{27}$$

with $\mathbf{H}_{n,n-1} = \mathbf{I}$ *and* $\mathbf{H}_{n,j} = \mathbf{\Phi}(n-1, j+1)\,\mathbf{\Sigma}_{j+1}$ *for* $j \le n - 2$*.*

Theorem 1. *Let Assumption 1 hold, and let the gains be such that*

$$\bar{e} := \bar{w} \sup_{n \ge 1} \sum_{j=0}^{n-1} \|\mathbf{H}_{n,j}\|, \qquad \delta^\star := \bar{w} \sup_{n \ge 2} \sum_{j=0}^{n-2} \|\mathbf{K}_n\mathbf{H}_{n,j}\| \tag{28}$$

are finite. Then, for every admissible disturbance sequence and all n*,*

1) $\|\mathbf{e}_n\| \le \bar{e}$ *and* $\|\Delta\mathbf{p}_n\| \le \delta^\star$*;*
2) *if the plan satisfies* (23) *for all steps with* $\delta \ge \delta^\star$*, the realized footholds, swing trajectories and swept regions satisfy* (13)*.*

Hence a plan returned by BOBSTP with $\delta \ge \delta^\star$ *satisfies requirements 2 and 3 of Problem 1.*

The theorem decouples the problem: the dynamics determine how much the footholds may have to move, and the geometry, through $\mathcal{O}_\delta$, absorbs that motion. The planner needs no balance model beyond the single number $\delta^\star$. For steady walking, the matrices alternate between the two pairs, $(\mathbf{\Sigma}_n, \mathbf{\Lambda}_n, \mathbf{K}_n) = (\mathbf{\Sigma}_A, \mathbf{\Lambda}_A, \mathbf{K}_A)$ and $(\mathbf{\Sigma}_B, \mathbf{\Lambda}_B, \mathbf{K}_B)$, so $\mathbf{M}_n$ alternates between $\mathbf{M}_A$ and $\mathbf{M}_B$. The sums in (28) are then finite if the two-step matrix is stable, $\rho(\mathbf{M}_B\mathbf{M}_A) < 1$, where $\rho(\cdot)$ is the spectral radius, and they are evaluated numerically by summing the impulse responses of both phases. A single step may well be unstable, $\rho(\mathbf{M}_A) > 1$, while the gait as a whole is stable.

In the scalar case $\mathbf{\Sigma}_n = \sigma\mathbf{I}$, $\mathbf{\Lambda}_n = \gamma\mathbf{I}$, $\mathbf{K}_n = K\mathbf{I}$, let $\rho := \sigma + \gamma K$. Then $\mathbf{H}_{n,j} = \sigma\rho^{n-2-j}\mathbf{I}$ for $j \le n - 2$, and for $|\rho| < 1$ the bounds take the closed form

$$\bar{e} = \bar{w}\Big(1 + \frac{|\sigma|}{1 - |\rho|}\Big), \qquad \delta^\star = \bar{w}\,\frac{|K\sigma|}{1 - |\rho|}, \tag{29}$$

which are attained asymptotically by suitable disturbance sequences. For the LIP values $\sigma = a$, $\gamma = 1 - a$, the condition $|\rho| < 1$ reads $1 < K < \frac{a+1}{a-1}$.

Corollary 1 (Fixed footholds)**.** *Let the footholds be fixed by the plan (*$\mathbf{K}_n = \mathbf{0}$*), and let the stance map alternate between two pairs as above. If* $\rho(\mathbf{\Sigma}_B\mathbf{\Sigma}_A) < 1$*, then* $\delta^\star = 0$ *and* $\bar{e} < \infty$*. If* $\rho(\mathbf{\Sigma}_B\mathbf{\Sigma}_A) > 1$*, then for every* $\bar{w} > 0$ *there is an admissible disturbance sequence for which* $\|\mathbf{e}_n\|$ *is unbounded, so requirement 3 of Problem 1 fails.*

Corollary 1 separates two regimes. If the stance controller alone stabilizes the error over a gait cycle, the original

planner with $\delta = 0$ already solves Problem 1. For the LIP, $\rho(\mathbf{\Sigma}_B\mathbf{\Sigma}_A) = a^2 > 1$ and fixed footholds fail; in practice the body gains of the stance controller determine the matrices. This is consistent with Fig. 8a and Fig. 8b, where the robot falls, and with the high gain $k = 10$ needed in Section V to keep it upright with fixed footholds. Foot placement extends the planner to the regime $\rho(\mathbf{\Sigma}_B\mathbf{\Sigma}_A) \geq 1$, for instance at higher speed or when disturbances saturate the stance controller, and it reduces the error bound $\bar{e}$ when the stance controller is stable but slow, at the price of the margin $\delta^\star$.

Corollary 2 (Deadbeat gain)**.** *If every $\mathbf{\Lambda}_n$ is invertible, the gains $\mathbf{K}_n = -\mathbf{\Lambda}_n^{-1}\mathbf{\Sigma}_n$ give $\mathbf{M}_n = \mathbf{0}$ and*

$$\bar{e} = \bar{w}\left(1 + \sup_n \|\mathbf{\Sigma}_n\|\right), \qquad \delta^\star = \bar{w}\sup_n \left\|\mathbf{\Lambda}_n^{-1}\mathbf{\Sigma}_n\mathbf{\Sigma}_{n-1}\right\|. \tag{30}$$

In the scalar case, this $K_{db} = -\sigma/\gamma$ minimizes $\bar{e}$ over all K with $|\rho| < 1$; it also minimizes $\delta^\star$ if $|\sigma| \geq 1$, with $\delta^\star_{\min} = \frac{\sigma^2}{|\gamma|}\bar{w}$, whereas for $|\sigma| < 1$ the margin is minimized by $K = 0$, with $\delta^\star = 0$.

By (30), an ill-conditioned $\mathbf{\Lambda}_n$, i.e., a direction in which shifting the footholds barely affects the DCM, makes the deadbeat margin large. In that case the gains are better chosen to minimize $\rho(\mathbf{M}_B\mathbf{M}_A)$ under a bound on $\|\mathbf{K}_n\|$, which trades the convergence rate against $\delta^\star$.

Remark 1. For the LIP model of the Mini Cheetah trot used in Section V, $h = 0.29\,\mathrm{m}$ and $T_s = 0.13\,\mathrm{s}$, so $a \approx 2.13$, $K_{\mathrm{db}} \approx 1.88$ and $\delta^\star_{\min} = \frac{a^2}{a-1}\bar{w} \approx 4.0\,\bar{w}$. The predictor (12) must use the matrices of the actual closed loop: if the LIP values are used on a robot whose stance controller cancels most of the correction, $\mathbf{\Lambda}_n \approx \mathbf{0}$, the correction obeys $\Delta\mathbf{p}_{n+1} \approx Ka\,\mathbf{e}_n - K(a-1)\Delta\mathbf{p}_n$, which diverges for every $K > 1/(a-1) \approx 0.89$, i.e., for every K admitted by the LIP analysis.

Remark 2. The velocity-only Raibert feedback used on the Mini Cheetah [21] can be analysed in the same way with a four-dimensional error state.

V. Experiments

A. Simulation Environment

The simulation environment that is used to test the validity of the control technique is developed by MIT specifically for the Cheetah family of quadruped robots [17], [23]. It provides a high-fidelity, physics-based platform that enables safe and repeatable experimentation with new control strategies before deployment on hardware. The simulator replicates the full-body dynamics of the robot, supports low-level torque control, and includes realistic sensor emulation, making it suitable for evaluating locomotion, trajectory tracking, and disturbance rejection performance.

In this work, the Mini Cheetah variant is used. The Mini Cheetah, first introduced in [6] and shown in Fig. 5, is a lightweight, low-cost quadruped robot designed to accelerate the development and testing of control algorithms. It features twelve actuated joints with backdrivable motors and onboard sensing capabilities, making it well-suited for agile locomotion tasks. Its open design and robust mechanical structure have made it a popular platform for research in legged locomotion and dynamic motion planning.

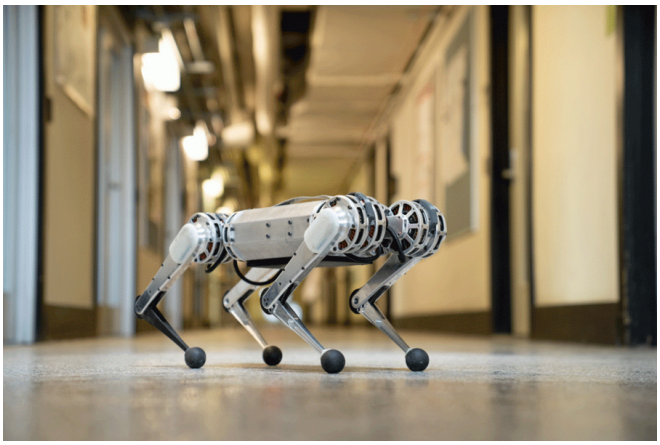


Fig. 5. The MIT Mini Cheetah robot, developed at the Biomimetic Robotics Laboratory, demonstrates advanced quadrupedal locomotion and agility. [22].

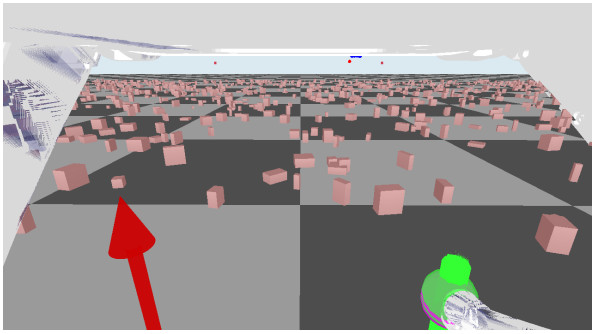


Fig. 6. Rendered view in the simulation of a fixed camera attached to the stomach of the robot, useful for analyzing the environment beneath the robot to detect obstacles and assess terrain features during locomotion.

To support development and evaluation, the OpenGL-based renderer [24] has been extended with multiple camera views, fixed in the world or on the robot (see Fig. 6), and a data recorder has been added that logs time-stamped state and control signals to CSV files and camera output to synchronized video.

B. Control Architecture

The existing Mini-Cheetah control system has been adapted to work with the BOBSTP algorithm and the tracking of the resulting path. The adapted control architecture is visualized in Fig. 7.

The control framework consists of a high-level controller responsible for determining the desired state of the robot's geometric center, denoted by $\boldsymbol{\xi}_{6\mathrm{D}}$ and $\mathbf{v}_{6\mathrm{D}}$. Here, the subscript $_{6\mathrm{D}}$ indicates that the variable represents the full 6D state, including the non-planar components of their planar counterparts. In addition to computing the desired center-of-mass (COM) state, the high-level controller also determines the appropriate gait pattern. In the high-level controller, the trajectory is determined by the BOBSTP and then followed

Fig. 7. Overview of the control architecture for the Mini-Cheetah's tree planner within the existing system. The diagram shows high-level controllers (green), leg and body locomotion modules (yellow), the robot with low-level motor control (blue), and state estimation components (purple). Dashed borders indicate existing modules, while solid borders mark newly introduced or significantly modified elements. GRF and WBC denote Ground Reaction Force and Whole-Body Controller, respectively.

by the trajectory tracking controller. Additionally, the estimated ground contact forces, for the stance legs, and the current estimated state of the robot are used.

The resulting COM state and gait parameters are then passed to the leg and body control modules, which compute the desired swing leg trajectories and the ground reaction forces (GRFs) to be applied by the stance legs. These quantities are subsequently processed by a whole-body controller, which resolves them into joint-level commands, specifically, desired joint positions, velocities, and torques, suitable for low-level execution.

Trajectory Tracking Controller: The mini cheetah software requires a state trajectory $\mathbf{q}(t)$ containing the desired planar pose $\boldsymbol{\xi}(t)$ and desired planar velocities $\mathbf{v}(t)$. As the BOBSTP algorithm returns a discrete-time state trajectory $\mathbf{q}_k$, the trajectory is converted to a continuous-time trajectory using a piecewise-constant backwards Euler reconstruction as

$$\mathbf{q}(t) = \begin{cases} \mathbf{q}_0, & t = 0, \\ \mathbf{q}_{k+1}, & t \in (k\Delta t,\ (k+1)\Delta t], \\ & \text{for all } k \in \{0, 1, \ldots, N-1\}. \end{cases} \tag{31}$$

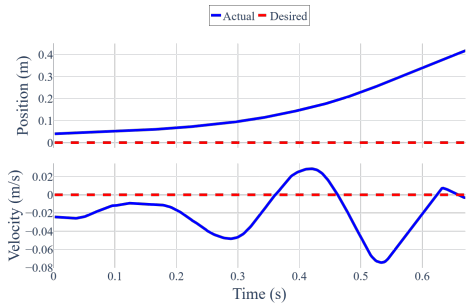
The nominal footholds and the planned DCM, in contrast, use the interpolated reconstruction (21).

The desired velocity, $\mathbf{v}_{des}$, is calculated with the PD control law

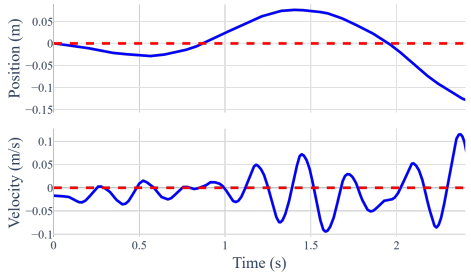
$$\mathbf{v}_{des} = \mathbf{v}_{\text{curr}} + \mathbf{K}_D\left(\mathbf{v}(t) - \mathbf{v}_{curr}\right) + \mathbf{K}_P\left(\boldsymbol{\xi}(t) - \boldsymbol{\xi}_{curr}\right), \tag{32}$$

where $\mathbf{K}_D$ and $\mathbf{K}_P$ are diagonal gain matrices associated with the derivative and proportional error terms, respectively.

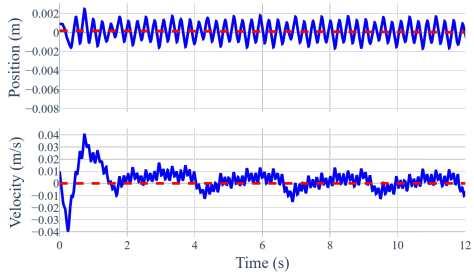
With constraints on foot placement, the stabilizing component of the step, that is, the use of swing legs to actively adjust footholds for maintaining or regaining balance, is not available. It is important to clarify that this stabilizing component is not limited to recovery from disturbances like external pushes but also includes any purposeful stepping action aimed at controlling the robot's balance or trajectory, distinct from merely moving toward a desired robot state. Because stepping for balance adjustment is prohibited, the entire responsibility for maintaining stability falls on the whole-body controller and additional compensation control laws. As shown in Fig. 8a, the existing whole-body controller is unable to maintain balance in all directions of the robot frame R, and particularly struggles to control the center of mass along the direction perpendicular to the robot's nominal trajectory. This limitation arises when foothold adjustments via the swing legs are disallowed, as when you have predetermined footholds calculated by the BOBSTP algorithm. A compensation term is introduced in both $\mathbf{K}_P$ and $\mathbf{K}_D$ to regulate motion along the y-axis of the robot frame R. As illustrated in Fig. 8, achieving stable behavior requires relatively high gain values, specifically $k = 10$ for both



(a) $\mathbf{K}_D = \text{diag}(6.0, 0.0, 2.0)$, $\mathbf{K}_P = \text{diag}(0.5, 0.0, 0.5)$



(b) $\mathbf{K}_D = \text{diag}(6.0, 1.0, 2.0)$, $\mathbf{K}_P = \text{diag}(0.5, 1.0, 0.5)$



(c) $\mathbf{K}_D = \text{diag}(6.0, 10.0, 2.0)$, $\mathbf{K}_P = \text{diag}(0.5, 10.0, 0.5)$

Fig. 8. Visualization of the behavior during a forward straight-line trajectory. Positions and velocities correspond to the y-axis of the robot frame R, showing the dynamic response. Fig. 8a and Fig. 8b are unstable, causing the robot to fall before reaching the goal, while Fig. 8c demonstrates stable balance control.

the proportional and derivative components. In Fig. 8c, the robot successfully follows the desired trajectory to the goal. The observed oscillations are consistent with the stepping frequency, indicating that the noise is a natural consequence of the gait cycle rather than instability. Additional compensation is also applied to the roll and pitch angles of the robot. The objective is to induce a forward-leaning posture in the direction of motion, thereby enhancing dynamic stability and improving contact consistency during locomotion.

Locomotion control: The locomotion control modules ensure that, given desired center of mass (CoM) state trajectories, appropriate joint-level commands are computed. These include joint angles, joint velocities, and joint torques required to realize the planned motion. The controller, along with the Whole-Body Impulse Controller (WBIC), was first introduced by MIT in [21]. It is structured into two main components: a ground reaction force controller for the stance feet and a swing leg controller for the swing phases, which are subsequently integrated within the WBIC to coordinate the overall motion of the robot. The swing leg controller has been adapted to implement the Adaptive Bézier Foot Trajectory as outlined in Section III-E.

The ground reaction force controller computes the contact forces for the stance feet using a convex Model Predictive Controller (cMPC), as presented in [4]. Despite relying on a highly simplified model of the robot's dynamics, representing the robot as a single rigid body without roll and pitch and neglecting leg dynamics, the optimization framework is capable of generating robust and stable locomotion. This simplification enables a convex formulation, allowing for a prediction horizon of up to 0.5 seconds, which can be solved to optimality in under 1 ms at a control rate of 20–30 Hz. A comprehensive description of the full formulation and implementation details can be found in [4].

The Whole-Body Impulse Controller (WBIC) combines the outputs of the ground reaction force (GRF) controller and the swing leg controller to compute the desired joint-level inputs. Originally introduced for the Mini-Cheetah quadruped robot in [6], WBIC differs from conventional Whole-Body Controllers (WBCs) that primarily track predefined body trajectories. Instead, it focuses on generating physically consistent ground reaction force commands, enabling agile and high-speed locomotion, including aerial phases. The control problem is formulated as a strictly convex quadratic program (QP), solved efficiently using an open-source solver [25]. This formulation allows the controller to enforce contact constraints, prioritize swing leg tracking, and compute joint torques and accelerations that are dynamically consistent. The resulting joint commands are executed through low-level torque controllers. Due to its computational efficiency, WBIC operates in real time and has been validated on the Mini-Cheetah robot in a range of dynamic locomotion tasks, including trotting, bounding, and jumping.

C. Experimental Setup

To validate the performance of the BOBSTP algorithm, a series of experiments is conducted in simulated environments with varying levels of complexity. The objective is to evaluate the algorithm's robustness and efficiency in cluttered settings. The validation is divided into two separate tests: (1) a feasibility test to determine whether the algorithm can successfully find a collision-free path, and (2) a stability test to assess whether the robot can follow the planned path while maintaining stable behavior. The primary performance metrics include success rate in planning, success rate in execution, path length, and computation time.

The robot operates in a square environment of size $6 \times 6\,\mathrm{m}$, centered at the origin. It begins from a fixed initial pose $\boldsymbol{\xi}_{\text{init}} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^\top$, and is tasked with reaching a goal position located at a distance of $4\,\mathrm{m}$. For each experimental condition, 40 goal positions are sampled uniformly on a circle of radius $4\,\mathrm{m}$ centered at $\mathbf{p}_{\text{init}}$, ensuring consistent challenge across trials.

Environmental complexity is introduced through the placement of static obstructions, modeled as cuboids with edge lengths randomly sampled between 2 and $10\,\mathrm{cm}$. These cuboids represent simplified plant-like obstacles. The chosen size range reflects the assumption that the robot is capable of stepping over small plants, rather than needing to fully navigate around them, as it is a local path planner. This ensures that, even at higher obstruction densities, feasible paths may still exist, allowing the algorithm's performance to be evaluated under increasingly constrained but not completely impassable conditions. Obstruction density is defined as the number of cuboids per square meter. For each trial, obstacles are randomly distributed throughout the environment, avoiding overlaps and initial collisions with the robot. A fixed random seed is used to ensure reproducibility. Fig. 6 illustrates an example environment with an obstruction density of $50\,\mathrm{m}^{-2}$, where obstacles are visualized as pink cuboids.

In the *Path Planning Experiments*, eight different obstruction densities are tested: 0, 10, 20, 30, 40, 50, 60, and $70\,\mathrm{m}^{-2}$. For each density, 40 independent trials are executed using the same set of goal positions. Two different parameter configurations are applied (see Table I). The first configuration uses *conservative exploration settings* ($N_{\text{max,samples}} = 10$, $\eta = 2.0$, $N_{\text{ancestor}} = 5$), which restrict the planner to grow its tree cautiously and locally. The second configuration employs *aggressive exploration settings* ($N_{\text{max,samples}} = 20$, $\eta = 5.0$, $N_{\text{ancestor}} = 4$), encouraging the planner to probe a broader region of the state space. Comparing these two regimes provides valuable insight: while the conservative parameters are expected to generate stable and reliable plans in simpler environments, the more aggressive exploration in the second setup may improve success rates in cluttered environments, at the cost of higher computational demand. The additional computational load arises because a larger pruning threshold allows the tree to grow denser, and the choice of N_{ancestor} sets how far up the tree search resumes after pruning.

A trial in this experiment is considered successful if a valid, collision-free path is found. At higher obstruction densities, it is possible that no feasible path exists, regardless of the planner configuration, which is not problematic since the experiment is meant to evaluate planner behavior, and the definition of success already accounts for feasibility. The experiment stops when either the planning tree reaches its maximum size of $N_{\text{tree,max}} = 1{,}000{,}000$, or when a total of five million nodes have been attempted. With the parameters in Table I, placing five million nodes requires approximately 10 seconds, corresponding to a maximum planning rate of 2.5 seconds per meter.

TABLE I

OVERVIEW OF THE PARAMETERS USED IN THE TWO-STEP PLANNER EXPERIMENTS. IF NO VALUE IS LISTED IN THE COLUMN OF AGGRESSIVE EXPLORATION CONFIGURATION, THE PARAMETER IS IDENTICAL TO THE CONSERVATIVE CONFIGURATION. THE TOLERANCE RADIUS $r_{\text{TOL,GOAL}}$ DEFINES THE ACCEPTABLE DISTANCE TO CONSIDER THE GOAL REACHED, t_{SIM} REFERS TO THE TIME OF ONE SIMULATION TICK, AND $N_{\text{TREE,MAX}}$ DENOTES THE MAXIMUM NUMBER OF NODES THAT CAN BE STORED IN THE PLANNING TREE.

Parameter	**Conservative**	**Aggressive**
Δt (s)	0.2	
p (-)	10	
$r_{\text{tol,goal}}$ (m)	0.02	
t_{sim} (s)	0.002	
$N_{\text{tree,max}}$ (-)	1,000,000	
$N_{\text{max,samples}}$ (-)	10	**20**
$n_{\text{nodes,iter}}$ (-)	1,000	
η (-)	2.0	**5.0**
N_{ancestor} (-)	5	**4**
$\mathbf{K}_P$ (-)	$\text{diag}(0.5, 10.0, 0.5)$	
$\mathbf{K}_D$ (-)	$\text{diag}(6.0, 10.0, 2.0)$	

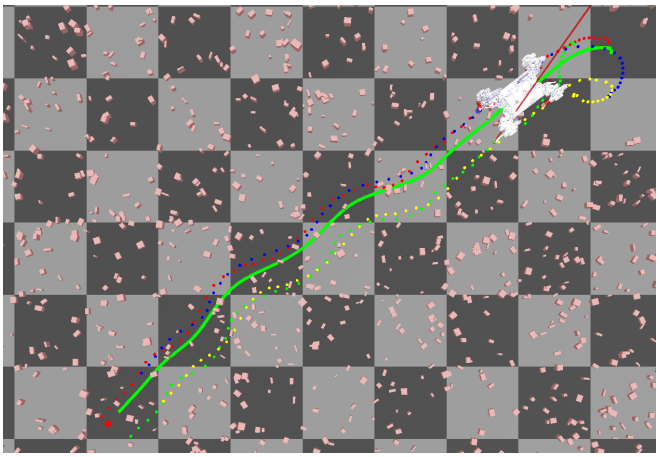


Fig. 9. Screenshot of a planned footstep path generated by the BOBSTP algorithm, illustrating obstacle avoidance (highlighted in pink). Footholds for each leg are color-coded: front right (red), front left (green), rear right (blue), and rear left (yellow). The planned path begins with an in-place rotation to align the robot toward the goal, followed by a sequence of steps that navigate around obstacles, deviating from a straight line as needed to avoid stepping on obstructions.

In the *Tracking Experiments*, the planned paths are executed by the robot. Importantly, these experiments only use the *successfully planned paths* from the feasibility tests. Thus, the tracking evaluation is independent of the parameter configuration used during planning. However, in practice, the second configuration is employed for execution, as it produces a higher number of successful plans, as shown later in the results. A trial is considered successful only if the robot remains stable and avoids all collisions during execution. Here, stability refers to *practical closed-loop tracking stability*, meaning that the position and orientation errors remain uniformly bounded and small, and no divergence or loss of balance occurs throughout the trial.

This two-phase evaluation, where different exploration settings are compared in the planning experiments and only successfully planned paths are used for the stability experiments, enables a clear assessment of how parameter choices affect planning feasibility, computational effort, and execution robustness under increasing environmental complexity.

To evaluate the impact of the adaptive Bézier foot trajectory, as introduced in Section III-E, a dedicated *Adaptive Bézier foot trajectory experiment* was conducted. The experiment compares the robot's performance using the previous foot trajectory implementation (baseline) and the new Bézier curve. The same path planned by the BOBSTP algorithm was executed under both configurations. Minor adjustments were made to the controller gains to optimize tracking performance with the new trajectory, ensuring that the comparison reflects the trajectory effect rather than suboptimal tuning.

The experiment was carried out by introducing a virtual obstacle positioned at the midpoint of the planned trajectory, with a height of $0.05\,\text{m}$, corresponding to half of the nominal foot swing height. The obstacle width was systematically increased from $0\,\text{cm}$ to $10\,\text{cm}$ in $1\,\text{cm}$ increments over approximately 70 trials. For each trial, the robot executed a single swing cycle over the obstacle while monitoring the foot's planned trajectory and actual position. The analysis focused on whether the foot trajectory adaptively increased clearance to avoid the obstacle, how accurately the actual foot followed the planned path, and whether the foot intersected the virtual obstacle. This setup allowed a detailed comparison of the baseline and new Bézier foot trajectories under progressively challenging conditions.

D. Results

Path Planning Experiment: For conservative configurations, the Fig. 10 shows that the planner's performance decreases with increasing obstacle density. The success rate, illustrated in Fig. 10a, remains high at low densities but declines sharply beyond $40\,\text{m}^{-2}$, reaching approximately 50% at $60\,\text{m}^{-2}$. A single failure at $20\,\text{m}^{-2}$ might be attributed to a local clustering of obstacles that prevented the algorithm from finding a path within the allotted node budget. A planned path with the corresponding footholds is shown for an obstacle density of $60\,\text{m}^{-2}$ is shown in Fig. 9.

Planning time increases with obstacle density, as shown in Fig. 10b. Without obstacles, a path is found within one simulation tick of $2\,\text{ms}$. For moderate densities between 10 and $40\,\text{m}^{-2}$, the median planning time remains near $0.1\,\text{s}$, although outliers of up to $5\,\text{s}$ raise the mean to near or above the 75th percentile. Both the mean and 75th percentile remain below $0.5\,\text{s}$ within this range. At $50\,\text{m}^{-2}$, the median time surpasses $1\,\text{s}$, and at $70\,\text{m}^{-2}$, 75% of successful runs conclude within $5\,\text{s}$. As planning time correlates with the total number of nodes evaluated, similar trends are observed in Fig. 10d.

Comparing the total number of nodes evaluated to the number of nodes retained in the tree, depicted in Fig. 10c, reveals that the tree contains substantially fewer nodes. At moderate obstacle densities, the difference is approximately

TABLE II

STANDARD DEVIATION OF POSITIONAL AND ROTATIONAL TRACKING ERRORS ACROSS SUCCESSFUL RUNS FOR DIFFERENT OBSTACLE DENSITIES (N_{PLANTS}).

Obstacle Density (m^{-2})	Positional std (mm)	Rotational std (rad)
0	0.654	0.0029
10	0.830	0.0039
20	0.597	0.0032
30	0.677	0.0033
40	0.622	0.0029
50	0.770	0.0030
60	0.907	0.0028
70	0.794	0.0037

an order of magnitude, increasing to two orders of magnitude at higher densities.

For the aggressive exploration configuration, a successful path was found for each obstacle density and each of the 40 runs, as shown in Fig. 11a. The planning time, visualized in Fig. 11b, for a 4-meter path increases with obstacle density, although the mean planning time remains below $0.5\,\mathrm{s}$, resulting in an average planning speed of $0.125\,\mathrm{s/m}$. For moderate densities, the maximum planning time also remains under one second; only in the extreme case of $70\,\mathrm{m}^{-2}$ does the maximum planning time exceed one second, reaching up to $4.6\,\mathrm{s}$. The total number of nodes evaluated exhibits a similar trend, as shown in Fig. 11d.

The number of nodes in the tree for the aggressive exploration configuration, shown in Fig. 11c, increases with obstacle density. Compared to the conservative configuration, the aggressive configuration requires significantly fewer nodes. Fig. 11c indicates that the relationship between obstacle density and the number of nodes in the resulting tree may be approximately exponential.

Tracking experiments: Across all successfully planned paths, the robot consistently tracked the desired trajectories in every run, demonstrating stable closed-loop performance. As shown in Fig. 12, the tracking *accuracy*, expressed by the mean and maximum errors, remained approximately constant across most obstacle density levels. The mean positional tracking error, measured as the Euclidean distance between the robot's geometric center and the desired geometric center along the path, was approximately $15\,\mathrm{mm}$ across all densities. In contrast, Table II highlights the tracking *precision*: the standard deviation across the 40 runs at each density level was less than $1\,\mathrm{mm}$ in position and $0.004\,\mathrm{rad}$ in orientation. Together, these results show that obstacle density did not affect either the accuracy or the precision of tracking for the majority of trials, indicating that the planned paths successfully avoided footholds on obstacles, which would otherwise have introduced deviations.

The maximum positional error was generally around $60\,\mathrm{mm}$, which can be attributed to the natural lateral swaying motion during walking. However, for the highest obstacle densities (60 and $70\,\mathrm{m}^{-2}$), the maximum positional error occasionally increased to approximately $70\,\mathrm{mm}$. Visual inspection revealed that in these rare cases, the robot stepped on an obstacle, though such events occurred very infrequently. For rotational tracking, the mean orientation error remained just below $0.02\,\mathrm{rad}$, with a maximum slightly above $0.1\,\mathrm{rad}$ across all densities.

Adaptive Bézier foot trajectory experiment: As shown in Fig. 13, the original MIT tuning of the foot trajectory resulted in a final placement error of approximately $2\,\mathrm{cm}$ after the swing-to-stance transition. By implementing the new adaptive Bézier trajectory along with controller gain adjustments, the final foot placement error was reduced to roughly $1\,\mathrm{cm}$, representing a twofold improvement. Across the entire swing cycle, the foot position tracking error was effectively halved compared to the baseline, indicating a more accurate and consistent execution of the planned trajectory. The trajectory is visualized for the old tuning in Fig. 14a and the new tuning in Fig. 14b.

In the case of the original trajectory, which does not account for obstructions, the quadruped was only able to successfully step over obstacles up to a width of approximately $5\,\mathrm{cm}$. With the adaptive Bézier trajectory, this capability increased, allowing the robot to reliably clear obstacles up to $8\,\mathrm{cm}$ in width, as illustrated in Fig. 14c. For obstacles of $9\,\mathrm{cm}$, however, the gait cycle was observed to interact with the obstacle, as shown in Fig. 14d. In this case, the actual foot trajectory exhibited a noticeable lateral deviation, skewing to the left of the planned path. It is important to note that while the executed foot path intersected with the obstacle for widths greater than $8\,\mathrm{cm}$, the adaptive trajectory planner did generate a collision-free reference path above the obstruction.

Robust foothold margin experiments: All runs in this section use the reconstruction (21) and walk $4\,\mathrm{m}$ to a goal on a circle, in 16 directions on clear ground. The obstacle runs use 8 directions on each of two terrain seeds.

LIP-based predictor. With the LIP values in (12), the robot fell in 6 of 16 runs at $0.1\,\mathrm{m/s}$ with $K = 1.01$, in 15 of 16 at $0.3\,\mathrm{m/s}$, and in all runs with the deadbeat gain. The DCM error stayed at a few millimetres while the correction grew by a factor of about -2.1 per step, as predicted by the remark in Section IV for a stance controller that cancels the correction.

Identification. The stance map (11) was identified by least squares from runs in which the footholds of each pair were shifted by a random offset, uniform in $[-15, 15]\,\mathrm{mm}$ per axis, independently of the error, with one pair of matrices per diagonal pair in the body frame. With the piecewise-constant reconstruction, the jumps of the reference dominated the residual: its median reference defect grew from $8\,\mathrm{mm}$ at $0.1\,\mathrm{m/s}$ to $40\,\mathrm{mm}$ at $0.5\,\mathrm{m/s}$. The interpolation (21) reduced it to $5\,\mathrm{mm}$ at $0.5\,\mathrm{m/s}$, and the identified model then explained the data to within 2–4 mm RMS, with a largest residual of 10–15 mm. Table III summarizes the identified maps. From $0.3\,\mathrm{m/s}$ on, each single step is unstable, but the gait cycle is stable, and it approaches the stability limit as the speed increases. The matrices $\mathbf{\Lambda}_n$ are ill-conditioned: in one direction, a foothold shift changes the DCM by only about $5\,\%$ of its size.

TABLE III

IDENTIFIED STANCE MAPS WITH INTERPOLATED REFERENCE AND THE DESIGNED GAINS: SPEED v IN M/S, LARGEST RESIDUAL $\bar{w} = \max_n \|\mathbf{w}_n\|$ AND MARGIN $\delta^\star$ IN MM.

v	$\rho(\mathbf{\Sigma}_A)$, $\rho(\mathbf{\Sigma}_B)$	$\rho(\mathbf{\Sigma}_B\mathbf{\Sigma}_A)$	$\rho(\mathbf{M}_B\mathbf{M}_A)$	$\sigma_{\min}(\mathbf{\Lambda}_n)$	$\bar{w}$	$\delta^\star$
0.1	1.00, 0.77	0.49	0.16	0.056	10	14
0.3	1.19, 1.19	0.85	0.41	0.050	14	44
0.5	1.22, 1.21	0.93	0.52	0.045	14	54
0.7*	1.21, 1.20	0.97	0.54	0.049	15	60

*One identification run, in which the robot fell, is excluded.

Fixed footholds and identified predictor. With fixed footholds, the robot fell in 1 of 16 runs at $0.7\,\mathrm{m/s}$, against 5 of 8 directions with the piecewise-constant reconstruction, consistent with Corollary 1 and a two-step spectral radius just below one. Because of the ill-conditioned $\mathbf{\Lambda}_n$, deadbeat gains are impractically large, so we chose gains with $\|\mathbf{K}_n\| \leq 1$ that minimize $\rho(\mathbf{M}_B\mathbf{M}_A)$. These roughly halve the two-step spectral radius, with $\delta^\star$ between $1.5\,\bar{w}$ at $0.1\,\mathrm{m/s}$ and $4\,\bar{w}$ at $0.7\,\mathrm{m/s}$ (Table III). With these gains, the robot did not fall in any clear-ground run. The largest DCM error per run, as a median over the directions, was unchanged at $0.1\,\mathrm{m/s}$ ($16\,\mathrm{mm}$), and decreased from 32 to $26\,\mathrm{mm}$ at $0.3\,\mathrm{m/s}$, from 48 to $36\,\mathrm{mm}$ at $0.5\,\mathrm{m/s}$, and from 62 to $45\,\mathrm{mm}$ at $0.7\,\mathrm{m/s}$.

Obstacles and swing tracking. The obstacle runs walk at $0.3\,\mathrm{m/s}$ with 5 and 10 obstacles per square metre. With fixed footholds ($\delta = 0$) and $\varepsilon = 10\,\mathrm{mm}$, the planner found a path in 29 of 32 runs, and every planned foothold kept at least ε from all obstacles, yet feet entered obstacles in 23 of these runs. Of 844 sampled foot positions inside an obstacle, 843 occurred while the foot deviated from its plan by more than $\delta + \varepsilon$: during the swing, or at lift-off after an earlier perturbed touchdown. The measured swing tracking error is about $31\,\mathrm{mm}$ on clear ground, so Assumption 1(ii) does not hold for $\varepsilon = 10\,\mathrm{mm}$. With $\varepsilon = 35\,\mathrm{mm}$, the planner found a path in 10 of 16 runs at 5 obstacles per square metre, and a foot touched an obstacle in only one of them; at 10 obstacles per square metre, it found a path in only 2 of 16 runs within the search budget. The identified predictor with its margin $\delta^\star = 44\,\mathrm{mm}$ and $\varepsilon = 10\,\mathrm{mm}$ removed all contacts, but the planner found a path in only 4 of 16 runs at 5 obstacles per square metre and in none at 10; with $\varepsilon = 35\,\mathrm{mm}$, it found none.

E. Discussion

Path Planning Experiment: The results demonstrate that the BOBSTP algorithm reliably generates feasible footstep paths across an aggressive range of obstacle densities. For an aggressive exploratory configuration, paths are found within sub-half-second planning times for all obstacle densities, indicating computational efficiency suitable for real-life applications. Considering that a planning horizon of approximately 4 meters is relatively large for a local planner, especially given the practical limitations of sensor accuracy at such distances, where visual perception and mapping become less reliable, these findings validate the suitability of the planner for realistic deployment scenarios.

At the highest obstacle density tested ($70\,\mathrm{m}^{-2}$), the environment represents an extreme edge case where the physical constraints and limited foothold availability make feasible path planning especially challenging for a quadruped robot. In such densely cluttered scenarios, a correctly tuned configuration is of importance as the conservative exploration configuration was unable to find feasible paths in less than 40% of the cases. In even denser, cluttered scenarios, alternative robotic platforms, such as legged robots with fewer legs or aerial vehicles, may be inherently better suited to navigate the environment effectively. This highlights the importance of selecting the appropriate robotic configuration based on the expected operational context and obstacle complexity.

An approximately exponential relationship between the number of nodes in the resulting tree and the obstacle density was observed. This trend suggests that, as the environment becomes more cluttered, the planner requires substantially more nodes to successfully explore feasible paths. Such an increase has practical implications: the total number of nodes directly affects both the memory usage and the computational load of the algorithm. With the current implementation, the number of stored nodes is limited by available memory, which could potentially constrain the maximum obstacle density that can be handled efficiently.

Looking forward, integrating the BOBSTP local planner with a global or high-level planner could further enhance path feasibility and robustness. A hierarchical planning approach, where the global planner strategically guides the robot away from highly dense obstacle regions, would reduce the burden on the local planner to resolve complex footstep placements in cluttered spaces. In this scheme, the local planner would focus on generating stable footholds while the global planner manages larger-scale path deviations and goal-reaching strategies. This layered approach is hypothesized to significantly improve the robot's capability to navigate complex real-world environments. This strategy would alleviate the burden on the BOBSTP local path planner, allowing it to concentrate its allotted node budget on precise foot placement rather than handling large deviations from the nominal path.

Additionally, systematic benchmarking of the BOBSTP algorithm against state-of-the-art footstep planners, as mentioned in Fig. I, is necessary to quantitatively establish its comparative advantages and limitations. Metrics such as planning success rate, computational time, path quality, and robustness to sensor noise should be considered in future studies.

Finally, exploring adaptive step sizes, variable gait patterns, and real-time sensor feedback integration could address current limitations observed at high obstacle densities. These enhancements would allow the planner to dynamically adjust to environmental complexity and robot stability constraints, thereby extending the planner's applicability and resilience.

Tracking experiments: The results of the tracking experiments demonstrate that the robot maintained consistent and robust closed-loop performance across all obstacle densities. The small standard deviations in positional ($< 1\,\mathrm{mm}$) and

rotational ($< 0.004\,\mathrm{rad}$) tracking errors for the geometric center of the robot indicate a high degree of run-to-run consistency, confirming the reliability of the control strategy.

Notably, tracking performance remained largely unaffected by increasing obstacle density, suggesting that the planned paths effectively avoided collisions with obstacles and that the robot's controller could compensate for minor deviations without loss of stability. The observed maximum positional error of approximately $60\,\mathrm{mm}$ can be attributed to natural lateral swaying during walking, while the rotational errors, with a mean below $0.02\,\mathrm{rad}$ and a maximum slightly above $0.1\,\mathrm{rad}$, remain within acceptable operational bounds. For the highest obstacle densities (60 and $70\,\mathrm{m}^{-2}$), rare occurrences of maximum positional errors up to $70\,\mathrm{mm}$ were observed, corresponding to occasional instances where the robot stepped on an obstacle. These events were infrequent and did not compromise overall tracking performance.

These findings highlight that, once a feasible path is planned, the execution phase is reliably handled by the controller. Therefore, the primary challenge lies in ensuring successful path planning in cluttered environments, as execution performance is robust across varying conditions. Future studies could extend these experiments to dynamic obstacles, uneven terrain, or higher walking speeds to further evaluate the limits of tracking stability under more challenging conditions.

Adaptive Bézier foot trajectory experiment: The results demonstrate that the proposed adaptive Bézier foot trajectory is capable of successfully planning collision-free motions over a broad range of obstructions. Although tracking performance was improved overall, for obstacles wider than approximately $8\,\mathrm{cm}$ and centered along the gait path, the execution accuracy was not sufficient to fully avoid interaction with the obstruction. This highlights a trade-off between improved clearance through trajectory adaptation and the increased tracking demands it places on the controller.

It should also be noted that the current experiments considered obstacles positioned at the midpoint of the swing trajectory. The observed left-skewed actual foot paths suggest that obstacle avoidance may be more effective when the obstruction is encountered later in the swing phase. Consequently, further experiments with obstacles placed at varying positions along the trajectory may extend the range of obstruction widths that can be successfully negotiated.

The observed tracking shortcomings may be attributed to the whole-body impulse controller, which introduces an additional control layer on top of the foot-level controller. While overall robot stability is essential, adjusting the weighting more toward foot accuracy could improve tracking without compromising stability. Furthermore, as shown in Fig. 14, tracking accuracy decreased when trajectories were adapted to avoid obstructions. With the current implementation, the z-trajectory and the xy-trajectory are generated independently. As a result, the vertical motion for lifting and lowering the foot is compressed into a reduced distance in the xy-plane, which leads to a shorter time span and thus abrupt height changes. A more effective strategy would be to allocate more time to the vertical transitions while allowing the middle portion of the motion in the xy-plane to proceed more quickly. This would keep the timing of the height changes consistent and result in smoother overall trajectories.

Another important consideration is the effect of controller gains on robustness. Higher feedback gains were employed to improve tracking accuracy, which proved stable in the flat-ground simulation environment used for this study. However, such high gains can reduce robustness to unmodeled dynamics and external disturbances, which are more prevalent in real-world scenarios. This implies that while the adaptive Bézier trajectory shows strong potential in simulation, additional tuning and robustness analysis will be required to ensure reliable performance during physical experiments.

Robust foothold margin experiments: The experiments show that the guarantee of Section IV depends on a model of the closed loop, not of the bare pendulum. The Mini Cheetah's stance controller makes each step unstable but the gait cycle stable at the tested speeds. Fixed footholds therefore suffice in steady walking, and a LIP-based correction destabilizes the robot. With the identified model, foot placement reduces the DCM error by about a quarter at speed and prevents the fall at $0.7\,\mathrm{m/s}$. The resulting margin of 3–4 $\bar{w}$ is commensurate with the obstacle sizes, so the gain trades the error bound against planning success: in the tested fields, the margin made planning mostly infeasible.

For obstacle avoidance, the planned paths kept the required clearance, and all but one contact was traced to swing tracking errors larger than ε. The guarantee is thus as good as the swing tracking. A realistic ε of $35\,\mathrm{mm}$ nearly removes all contacts at moderate densities, but makes planning infeasible at high densities. Improving swing tracking, for instance with feed-forward torques, which are disabled in the current controller, is therefore the main lever for using the margin in dense fields.

VI. Conclusion

This paper presented a novel local footstep planning approach for quadrupedal robots operating in obstacle-dense agricultural environments, addressing a relatively limited research field for quadrupeds. By explicitly considering both the robot's dynamical constraints and the existing obstacles, the proposed Best-first Online Backtracking State Tree Planner (BOBSTP) efficiently generates feasible velocity paths with collision-free foothold sequences. Complementing the planner, the adaptive Bézier curve method ensures that foot trajectories are smooth and dynamically adjusted to clear obstacles of varying heights, enhancing locomotion safety and stability. Simulation results demonstrated the effectiveness of this combined approach in navigating through irregular plant layouts while maintaining feasibility and robustness. The robust foothold margin certifies the plan under balance corrections for an identified model of the closed-loop stance dynamics; for the Mini Cheetah, this model shows that fixed footholds suffice in steady walking, and that the remaining obstacle contacts stem from swing tracking errors beyond the assumed bound.

Recommendation for future work

For future work, several directions are recommended. Firstly, replacing reliance on ground-truth obstacle information with real-time sensory input from cameras and depth sensors would allow the planner to perceive and map obstacles autonomously, enabling local footstep planning to adapt to real agricultural environments. Second, integrating global path optimization with the local footstep planner could improve overall navigation efficiency by guiding the robot along a strategically chosen route, while the local planner ensures precise and safe foot placement around obstacles. This hierarchical combination would allow the robot to make more informed long-term decisions without compromising local adaptability in complex terrains. Third, more research can focus on systematically comparing existing footstep planners, evaluating their performance in terms of feasibility, robustness, and adaptability, an analysis that has not been comprehensively conducted for quadrupedal footstep planners. Fourth, improving the swing-leg tracking would allow a smaller tracking bound ε, and thereby the robust margin to be used at high obstacle densities; a correction law that remains valid beyond the identified range of the stance map would make foot placement more useful for balance. Finally, experimental validation on a physical quadruped in real agricultural fields would provide critical insights into system performance and robustness in complex, real-world environments. In addition, further investigation of the adaptive Bézier foot trajectory should focus on obstacle placements at different phases of the swing cycle and on evaluating robustness under real-world disturbances to better understand its practical applicability.

APPENDIX I
PROOFS OF SECTION IV

Proof of Lemma 1. By (12) and (11), $\hat{\mathbf{e}}_{n+1} = \mathbf{e}_{n+1} - \mathbf{w}_n$ and $\Delta\mathbf{p}_{n+1} = \mathbf{K}_{n+1}\hat{\mathbf{e}}_{n+1}$ for $n \geq 0$, with $\hat{\mathbf{e}}_1 = \mathbf{0}$ since $\mathbf{e}_0 = \Delta\mathbf{p}_0 = \mathbf{0}$. Substituting into the definition of $\hat{\mathbf{e}}_{n+2}$ gives

$$\hat{\mathbf{e}}_{n+2} = \mathbf{M}_{n+1}\,\hat{\mathbf{e}}_{n+1} + \mathbf{\Sigma}_{n+1}\mathbf{w}_n, \qquad n \geq 0, \tag{33}$$

so that $\hat{\mathbf{e}}_n = \sum_{j=0}^{n-2} \mathbf{\Phi}(n-1, j+1)\,\mathbf{\Sigma}_{j+1}\mathbf{w}_j$. The first identity in (27) follows from $\mathbf{e}_n = \hat{\mathbf{e}}_n + \mathbf{w}_{n-1}$, and the second from $\Delta\mathbf{p}_n = \mathbf{K}_n\hat{\mathbf{e}}_n$. □

Proof of Theorem 1. 1) By Lemma 1 and the triangle inequality, $\|\mathbf{e}_n\| \leq \sum_j \|\mathbf{H}_{n,j}\|\,\|\mathbf{w}_j\| \leq \bar{e}$, and similarly $\|\Delta\mathbf{p}_n\| \leq \sum_j \|\mathbf{K}_n\mathbf{H}_{n,j}\|\,\|\mathbf{w}_j\| \leq \delta^\star$.

2) Footholds: $\bar{\mathbf{p}}_{i,n+1} = \bar{\mathbf{p}}^\star_{i,n+1} + [\,\Delta\mathbf{p}_{n+1}^\top \quad 0\,]^\top$ with $\|\Delta\mathbf{p}_{n+1}\| \leq \delta^\star \leq \delta$. If $\bar{\mathbf{p}}_{i,n+1} \in \mathcal{O}$, then $\bar{\mathbf{p}}^\star_{i,n+1} \in \mathcal{O} \oplus (\mathcal{B}^2_\delta \times \{0\}) \subseteq \mathcal{O}_\delta$, since balls are symmetric, contradicting (23a). Swings: by (6), the realized horizontal path differs from the nominal one by $(1 - B(\phi))\Delta\mathbf{p}_{n-1} + B(\phi)\Delta\mathbf{p}_{n+1}$, a convex combination of vectors of norm at most $\delta^\star$, hence itself of norm at most $\delta^\star$; the height profile is unchanged by Assumption 1(ii), and the tracking error adds at most ε. Every point of the executed swing therefore lies in $\bar{\mathbf{p}}^\star_{\text{swing},i}(\phi) \oplus (\mathcal{B}^2_\delta \times \{0\}) \oplus \mathcal{B}^3_\varepsilon$, which by (23c) does not meet $\mathcal{O}$. Swept regions: each point of (24) built from the realized footholds is shifted horizontally by $(1-s)\Delta\mathbf{p}_{n+1} + s\Delta\mathbf{p}_{n-1}$, again of norm at most $\delta^\star$, and (23b) gives the claim.

Scalar case: $\mathbf{\Phi}(n-1, j+1) = \rho^{\,n-2-j}\mathbf{I}$, so $\sum_j \|\mathbf{H}_{n,j}\| \leq 1 + |\sigma| \sum_{\ell \geq 0} |\rho|^\ell$ and $\sum_j \|\mathbf{K}_n\mathbf{H}_{n,j}\| \leq |K\sigma| \sum_{\ell \geq 0} |\rho|^\ell$, which gives (29). Choosing $\mathbf{w}_j = \bar{w}\,\operatorname{sign}(\sigma\rho^{n-2-j})\,\vec{d}$ for a fixed unit vector $\vec{d}$ makes the partial sums converge to these values as $n \to \infty$. □

Proof of Corollary 1. For $\mathbf{K}_n = \mathbf{0}$, $\mathbf{M}_n = \mathbf{\Sigma}_n$ and $\delta^\star = 0$. If $\rho(\mathbf{\Sigma}_B\mathbf{\Sigma}_A) < 1$, then $\|(\mathbf{\Sigma}_B\mathbf{\Sigma}_A)^m\| \leq c\lambda^m$ for some $c > 0$ and $\lambda < 1$, so $\|\mathbf{\Phi}(k, m)\| \leq c'\lambda^{(k-m)/2}$ and the first sum in (28) is bounded by a convergent geometric series. If $\rho(\mathbf{\Sigma}_B\mathbf{\Sigma}_A) > 1$, let $\mathbf{w}_0 \neq \mathbf{0}$ with $\|\mathbf{w}_0\| \leq \bar{w}$ have a nonzero component in the unstable invariant subspace of $\mathbf{\Sigma}_2\mathbf{\Sigma}_1$, whose spectral radius equals that of $\mathbf{\Sigma}_B\mathbf{\Sigma}_A$, and let $\mathbf{w}_j = \mathbf{0}$ for $j \geq 1$. Then $\mathbf{e}_{2m+1} = (\mathbf{\Sigma}_2\mathbf{\Sigma}_1)^m\mathbf{w}_0$ by Lemma 1, which is unbounded in m. □

Proof of Corollary 2. With $\mathbf{M}_n = \mathbf{0}$, $\mathbf{\Phi}(n-1, j+1) = \mathbf{0}$ for $j \leq n-3$, so $\mathbf{e}_n = \mathbf{w}_{n-1} + \mathbf{\Sigma}_{n-1}\mathbf{w}_{n-2}$ and $\Delta\mathbf{p}_n = \mathbf{K}_n\mathbf{\Sigma}_{n-1}\mathbf{w}_{n-2} = -\mathbf{\Lambda}_n^{-1}\mathbf{\Sigma}_n\mathbf{\Sigma}_{n-1}\mathbf{w}_{n-2}$, which gives (30).

Scalar case: for $\gamma \neq 0$, the map $K \mapsto \rho = \sigma + \gamma K$ is a bijection, so we minimize over $\rho \in (-1, 1)$ with $K = (\rho - \sigma)/\gamma$. The error bound (29) increases in $|\rho|$ and is minimal at $\rho = 0$, i.e., $K = K_{\text{db}}$. For the margin, $\delta^\star/\bar{w} = \frac{|\sigma|}{|\gamma|}h(\rho)$ with $h(\rho) := \frac{|\rho - \sigma|}{1 - |\rho|}$. If $|\sigma| < 1$, then $\rho = \sigma$ ($K = 0$) gives $h = 0$. If $|\sigma| \geq 1$, assume $\sigma \geq 1$ (the case $\sigma \leq -1$ follows by symmetry); then $|\rho - \sigma| = \sigma - \rho$, so $h(\rho) = \frac{\sigma - \rho}{1 - \rho}$ is nondecreasing on $[0, 1)$, since its derivative is $\frac{\sigma - 1}{(1-\rho)^2} \geq 0$, and $h(\rho) = \frac{\sigma - \rho}{1 + \rho}$ is decreasing on $(-1, 0]$, since its derivative is $-\frac{1 + \sigma}{(1+\rho)^2} < 0$. Hence h is minimal at $\rho = 0$, where $h = \sigma$ and $\delta^\star = \sigma^2\bar{w}/|\gamma|$. □

APPENDIX II
STABILITY ANALYSIS VIA LASALLE'S INVARIANCE PRINCIPLE

This section analyses the stability of the closed-loop control system by employing a discrete-time Lyapunov function and LaSalle's invariance principle. The system state is shown to converge asymptotically to the desired goal position and orientation under suitable conditions on the control parameters and sampling time.

Define the error vector as

$$\mathbf{e}_k = \begin{bmatrix} x_k - x_G \\ y_k - y_G \end{bmatrix} = \boldsymbol{\xi}_k - \mathbf{p}_G, \tag{34}$$

considering only the positional error. A candidate Lyapunov function is given by

$$V(\mathbf{e}_k,\ \tilde{\theta}_{e,k}) = \frac{1}{2}\mathbf{e}_k^\top\mathbf{e}_k + \frac{c}{2}\tilde{\theta}^2_{e,k}, \tag{35}$$

where $\tilde{\theta}_{e,k}$ is the yaw error normalized to $(-\pi, \pi]$ as defined in (16) and $c > 0$ is a weighting scalar to balance the importance of heading error. The function is positive definite

and radially unbounded. In discrete time, LaSalle's condition becomes

$$\Delta V(\mathbf{e}_k,\, \tilde{\theta}_{e,k}) = V(\mathbf{e}_{k+1},\, \tilde{\theta}_{e,k+1}) - V(\mathbf{e}_k,\, \tilde{\theta}_{e,k}) \leq 0 \quad \forall \quad \mathbf{e}_k > 0 \quad \wedge \quad \tilde{\theta}_{e,k} > 0. \tag{36}$$

Taking the discrete-time linearized model as defined in (15), the error at the next time step is given by

$$\mathbf{e}_{k+1} = \boldsymbol{\xi}_{k+1} - \mathbf{p}_G = \mathbf{e}_k + \Delta t\, \mathbf{X}(\theta_k)\left(\mathbf{v}_k + \Delta t\, \mathbf{a}_{k+1}\right). \tag{37}$$

Substituting this into the Lyapunov candidate yields the updated value

$$\begin{aligned} V(\mathbf{e}_k,\, \tilde{\theta}_{e,k}) = \frac{1}{2}\mathbf{e}_k^\top \mathbf{e}_k + \frac{1}{2}\Delta t^2 \mathbf{v}_k^\top \mathbf{v}_k + \frac{1}{2}\Delta t^4 \mathbf{a}_{k+1}^\top \mathbf{a}_{k+1} \\ + \Delta t\, \mathbf{X}(\theta_k)\left(\mathbf{v}_k^\top + \Delta t\, \mathbf{a}_{k+1}^\top\right)\mathbf{e}_k \\ + \Delta t^3 \mathbf{v}_k^\top \mathbf{a}_{k+1} + \frac{c}{2}\tilde{\theta}_{e,k+1}^2. \end{aligned}$$

With $\mathbf{a}_{k+1} = \mathbf{a}_{k+1,\text{nom}}$ as defined in Section III-B, LaSalle's condition expands to

$$\begin{aligned} \Delta V(\mathbf{e}_k,\, \tilde{\theta}_{e,k}) = & \frac{1}{2}\Delta t^2 \mathbf{v}_k^\top \mathbf{v}_k \\ & + \frac{1}{2}\Delta t^4 \mathbf{a}_{k+1,\text{nom}}^\top \mathbf{a}_{k+1,\text{nom}} \\ & + \Delta t \mathbf{X}(\theta_k)\left(\mathbf{v}_k^\top + \Delta t \mathbf{a}_{k+1,\text{nom}}^\top\right)\mathbf{e}_k \\ & + \Delta t^3 \mathbf{v}_k^\top \mathbf{a}_{k+1,\text{nom}} \\ & + \frac{c}{2}\left(\tilde{\theta}_{e,k+1}^2 - \tilde{\theta}_{e,k}^2\right). \end{aligned}$$

By substituting from (**??**), the change in the Lyapunov function can be expressed as

$$\begin{aligned} \Delta V(\mathbf{e}_k,\, \tilde{\theta}_{e,k}) = & \underbrace{\frac{1}{2}\Delta t^2 \mathbf{v}_{k+1,des}^\top \mathbf{v}_{k+1,des}}_{\geq 0} \\ & + \Delta t\, \mathbf{X}(\theta_k)\mathbf{v}_{k+1,des}^\top \mathbf{e}_k \\ & + \frac{c}{2}\left(\tilde{\theta}_{e,k+1}^2 - \tilde{\theta}_{e,k}^2\right). \end{aligned} \tag{38}$$

While the first term is non-negative, its effect on the Lyapunov function can be limited by choosing a small Δt.

Focusing on the term involving the yaw error, the control input for the yaw rate $\omega_{k+1,des}$, as defined in (16), is designed to reduce the yaw error $\tilde{\theta}_{e,k}$ at each step. Consequently, the discrete-time difference of the squared yaw error satisfies the inequality

$$\frac{c}{2}\left(\tilde{\theta}_{e,k+1}^2 - \tilde{\theta}_{e,k}^2\right) \leq 0, \tag{39}$$

indicating that the yaw error decreases monotonically or remains constant and is, thus, non-positive.

The term containing the error vector can be rewritten by projecting the error vector onto the x-axis of the robot frame R. Noting that $\mathbf{v}_{k+1,des}$ lacks a y-component due to the non-holonomic constraint and using (17), this term becomes

$$\Delta t\, \mathbf{X}(\theta_k)\mathbf{v}_{k+1,des}^\top \mathbf{e}_k = \Delta t \mathbf{K}_{vel}\left(\mathbf{X}(\theta_k)\mathbf{e}_k\right)^\top \vec{x}\, v_{k+1,des},$$

where $\vec{x} = \begin{bmatrix}1 & 0 & 0\end{bmatrix}^\top$ denotes the unit vector in the direction of the x-axis in the world frame W, and $\mathbf{K}_{vel}$ is a diagonal positive-definite gain matrix. The linear velocity, as defined in (16), is non-negative and can be expressed with the same projection of the error vector as

$$v_{k+1,des} = \begin{cases} -\frac{\alpha_k}{\Delta t}\left(\mathbf{X}(\theta_k)\mathbf{e}_k\right)^\top \vec{x} & \text{if } \left(\mathbf{X}(\theta_k)\mathbf{e}_k\right)^\top \vec{x} < 0 \\ 0 & \text{elsewhere,} \end{cases} \tag{40}$$

where it must be noted that $\alpha_k \geq 0$ and $\Delta t \geq 0$. By recognizing that $v_{k+1,des}$ is positive if and only if the x-component of the projected error vector is negative, the concerning term is proven to be non-positive.

By selecting a sufficiently small time step Δt, sufficiently small gains in $\mathbf{K}_{vel}$, and an appropriately large weighting scalar c, the change in the Lyapunov function $\Delta V(\mathbf{e}_k)$ is guaranteed to be non-positive, ensuring stability of the closed-loop system. According to LaSalle's invariance principle, the system trajectories will converge to the largest invariant set within

$$\begin{aligned} \left\{\mathbf{e}_k, \tilde{\theta}_{e,k} \mid \Delta V(\mathbf{e}_k, \tilde{\theta}_{e,k}) = 0\right\} = \\ \left\{\mathbf{e}_k = \mathbf{0}, \quad \tilde{\theta}_{e,k} = 0\right\}. \end{aligned} \tag{41}$$

Thus, the state asymptotically converges to the desired goal position and orientation.

By relaxing the initial LaSalle's condition in (36) to allow for a tolerance region around the goal position $\mathbf{p}_G$, the formulation obtained is

$$\Delta V(\mathbf{e}_k,\, \tilde{\theta}_{e,k}) = V(\mathbf{e}_{k+1},\, \tilde{\theta}_{e,k+1}) - V(\mathbf{e}_k,\, \tilde{\theta}_{e,k}) \leq 0 \quad \forall \quad \|\mathbf{e}_k\| > r \quad \wedge \quad \tilde{\theta}_{e,k} > 0, \tag{42}$$

where $r > 0$ specifies the radius of this tolerance region. Within this neighbourhood, small residual errors or measurement noise are tolerated, and the Lyapunov function is not required to strictly decrease. This relaxation permits the use of a larger and more practical time step Δt, and facilitates tuning of the gain matrix to achieve a more responsive control behavior. As a result, the system becomes more robust to noise and discretization effects, without compromising overall stability.

REFERENCES

[1] R. Beumer, T. Janssen, R. van de Molengraft, and D. Antunes, "Mechanical precision weeding with a quadruped robot," 2026, arXiv preprint arXiv:2609.20048. [Online]. Available: https://arxiv.org/abs/2609.20048

[2] R. Beumer, S. Doodeman, R. van de Molengraft, and D. Antunes, "Semantic SLAM in precision agriculture using Bayesian inference," 2026, arXiv preprint arXiv:2609.20604. [Online]. Available: https://arxiv.org/abs/2609.20604

[3] M. Hutter, C. Gehring, A. Lauber, F. Gunther, C. D. Bellicoso, V. Tsounis, P. Fankhauser, R. Diethelm, S. Bachmann, M. Bloesch, H. Kolvenbach, M. Bjelonic, L. Isler, and K. Meyer, "ANYmal - toward legged robots for harsh environments," *Advanced Robotics*, vol. 31, no. 17, pp. 918–931, 9 2017. [Online]. Available: https://doi.org/10.1080/01691864.2017.1378591

[4] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the MIT Cheetah 3 through Convex Model-Predictive Control," *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 10 2018. [Online]. Available: https://doi.org/10.1109/iros.2018.8594448

[5] D. Kim, J. D. Carlo, B. Katz, G. Bledt, and S. Kim, "Highly dynamic quadruped locomotion via whole-body impulse control and model predictive control," *CoRR*, vol. abs/1909.06586, 2019, also cited as arXiv preprint arXiv:1909.06586. [Online]. Available: http://arxiv.org/abs/1909.06586

[6] B. Katz, J. Di Carlo, and S. Kim, "Mini Cheetah: a platform for pushing the limits of dynamic quadruped control," *2022 International Conference on Robotics and Automation (ICRA)*, 5 2019. [Online]. Available: https://doi.org/10.1109/icra.2019.8793865

[7] M. H. Raibert and E. R. Tello, "Legged robots that balance," *IEEE Expert*, vol. 1, no. 4, p. 89, 11 1986. [Online]. Available: https://doi.org/10.1109/mex.1986.4307016

[8] J. Pratt, J. Carff, S. Drakunov, and A. Goswami, "Capture Point: A Step toward Humanoid Push Recovery," *IEEE-RAS International Conference on Humanoid Robots*, vol. 6th, pp. 200–207, 12 2006. [Online]. Available: https://doi.org/10.1109/ichr.2006.321385

[9] T. Koolen, T. De Boer, J. Rebula, A. Goswami, and J. Pratt, "Capturability-based analysis and control of legged locomotion, Part 1: Theory and application to three simple gait models," *The International Journal of Robotics Research*, vol. 31, no. 9, pp. 1094–1113, 7 2012. [Online]. Available: https://doi.org/10.1177/0278364912452673

[10] S. Fasano, J. Foster, S. Bertrand, C. DeBuys, and R. Griffin, "Efficient, dynamic locomotion through step placement with straight legs and rolling contacts," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 1143–1150.

[11] V. Morlando, J. Cacace, and F. Ruggiero, "Online feet Potential fields for quadruped robots navigation in harsh terrains," *Robotics*, vol. 12, no. 3, p. 86, 6 2023. [Online]. Available: https://doi.org/10.3390/robotics12030086

[12] B. Mitchell, A. G. Hofmann, and B. C. Williams, "Search-based foot placement for quadrupedal traversal of challenging terrain," *Proceedings - IEEE International Conference on Robotics and Automation/Proceedings*, pp. 1461–1466, 4 2007. [Online]. Available: https://doi.org/10.1109/robot.2007.363190

[13] J. Chestnutt, M. Lau, G. Cheung, J. Kuffner, J. Hodgins, and T. Kanade, "Footstep planning for the honda asimo humanoid," in *Proceedings of the 2005 IEEE International Conference on Robotics and Automation*, 2005, pp. 629–634.

[14] R. J. Griffin, G. Wiedebach, S. McCrory, S. Bertrand, I. Lee, and J. Pratt, "Footstep planning for autonomous walking over rough terrain," in *2019 IEEE-RAS 19th International Conference on Humanoid Robots (Humanoids)*, 2019, pp. 9–16.

[15] V. Ranganeni, O. Salzman, and M. Likhachev, "Effective footstep planning for humanoids using Homotopy-Class guidance," *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 28, pp. 500–508, 6 2018. [Online]. Available: https://doi.org/10.1609/icaps.v28i1.13922

[16] R. Deits and R. Tedrake, "Footstep planning on uneven terrain with mixed-integer convex optimization," *IEEE-RAS International Conference on Humanoid Robots*, 11 2014. [Online]. Available: https://doi.org/10.1109/humanoids.2014.7041373

[17] J. J. Di Carlo, "Software and control design for the MIT cheetah quadruped robots," Master of Engineering Thesis, Massachusetts Institute of Technology, Cambridge, MA, Feb. 2020, department of Electrical Engineering and Computer Science; includes bibliographical references (pages 99–101). [Online]. Available: https://hdl.handle.net/1721.1/129877

[18] X. Zeng, S. Zhang, H. Zhang, X. Li, H. Zhou, and Y. Fu, "Leg Trajectory Planning for Quadruped Robots with High-Speed Trot Gait," *Applied Sciences*, vol. 9, no. 7, p. 1508, 4 2019. [Online]. Available: https://doi.org/10.3390/app9071508

[19] F. Risbourg, T. Corbères, P.-A. Léziart, T. Flayols, N. Mansard, and S. Tonneau, "Real-time footstep planning and control of the solo quadruped robot in 3D environments," *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 10 2022. [Online]. Available: https://doi.org/10.1109/iros47612.2022.9981539

[20] R. M. Beumer, C. R. A. Smeets, M. J. T. C. van Zutphen, and D. J. Antunes, "A novel quadruped path planner for obstruction-free foothold placement with applications to precision farming (extended version)," 2026, arXiv preprint arXiv:XXXX.XXXXX.

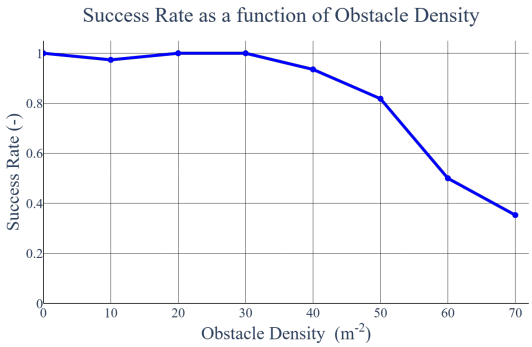
[21] G. Bledt, M. J. Powell, B. Katz, J. Di Carlo, P. M. Wensing, and S. Kim, "MIT Cheetah 3: Design and control of a robust, dynamic quadruped robot," *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 10 2018. [Online]. Available: https://doi.org/10.1109/iros.2018.8593885

[22] B. Vickmark, "Mit's new mini cheetah robot is springy, light on its feet, and weighs in at just 20 pounds," Photo in "Mini cheetah is the first four-legged robot to do a backflip" — MIT News, Mar. 2019, downloaded from MIT News website (Creative Commons Attribution-NonCommercial-NoDerivatives).

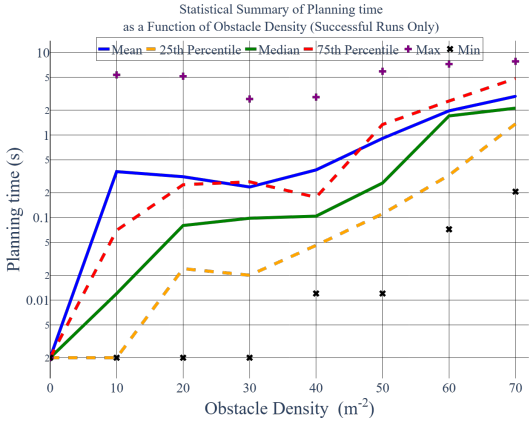
[23] MIT Biomimetic Robotics Lab, "Cheetah software," https://github.com/mit-biomimetics/Cheetah-Software, 2020, gitHub repository maintained under the mit-biomimetics organization. [Online]. Available: https://github.com/mit-biomimetics/Cheetah-Software

[24] The Khronos Group, *OpenGL 4.6 Core Profile Specification*, Khronos Group, 2017, https://www.khronos.org/registry/OpenGL/specs/gl/glspec46.core.pdf.

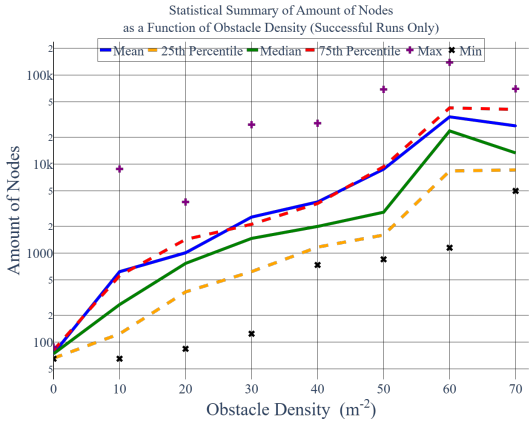
[25] D. Goldfarb and A. Idnani, "A numerically stable dual method for solving strictly convex quadratic programs," *Mathematical Programming*, vol. 27, no. 1, pp. 1–33, 9 1983. [Online]. Available: https://doi.org/10.1007/bf02591962



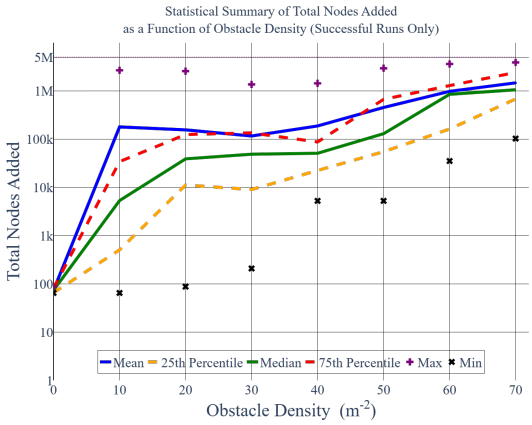
(a) Analysis of the success rate



(b) Planning time as a function of obstacle density, with a logarithmic scale on the y-axis.

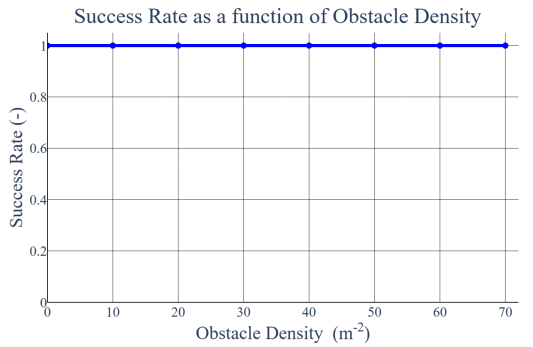


(c) Number of nodes in the tree at the moment a path is found, with a logarithmic scale on the y-axis.

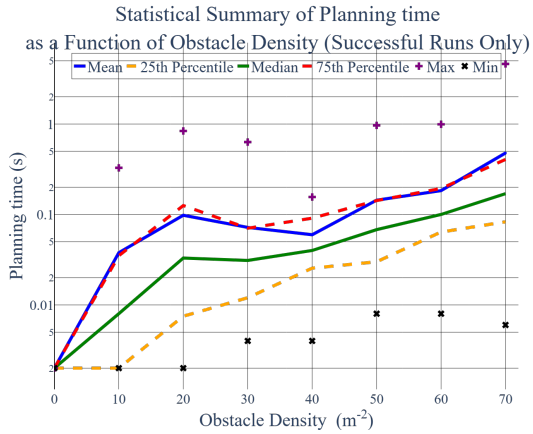


(d) Total number of nodes evaluated in the search for a path, with a logarithmic scale on the y-axis.

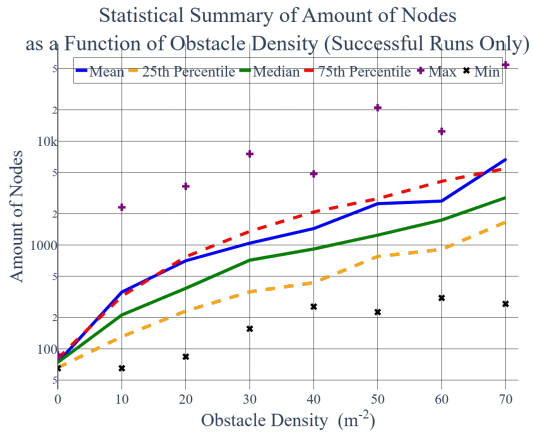
Fig. 10. Results for the path planning Experiment with multiple variables analyzed over an increasing obstacle density for the *conservative* exploration configuration.



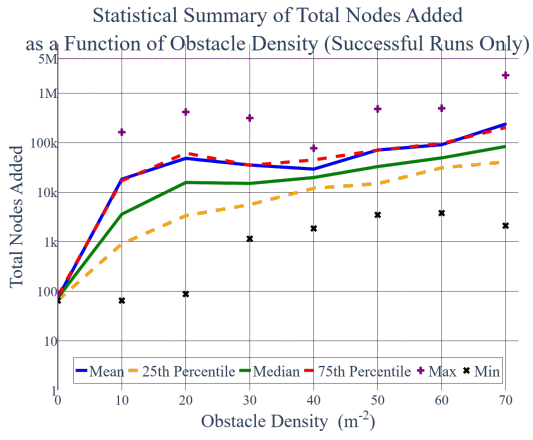
(a) Analysis of the success rate



(b) Planning time as a function of obstacle density, with a logarithmic scale on the y-axis.

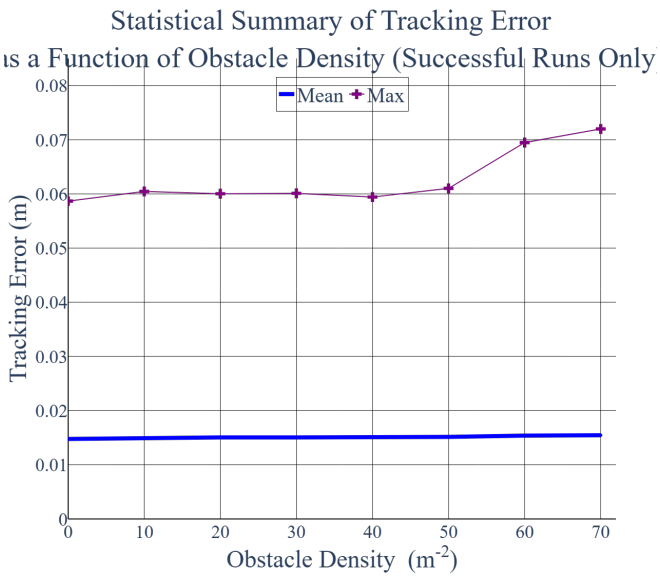


(c) Number of nodes in the tree at the moment a path is found, with a logarithmic scale on the y-axis.

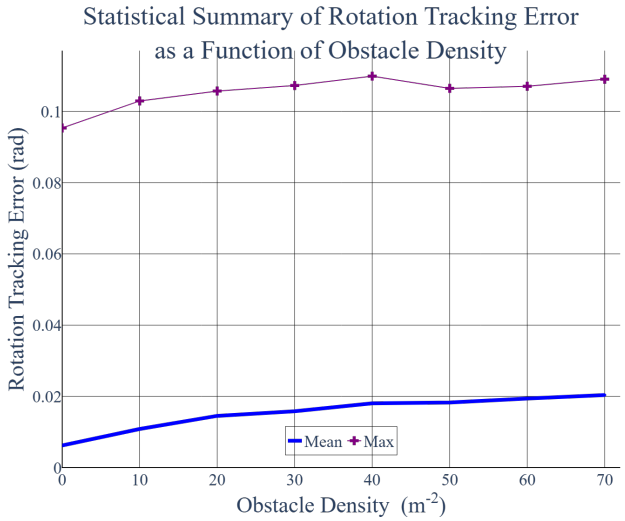


(d) Total number of nodes evaluated in the search for a path, with a logarithmic scale on the y-axis.

Fig. 11. Results for the path planning experiment with multiple variables analyzed over an increasing obstacle density for the *aggressive* exploration

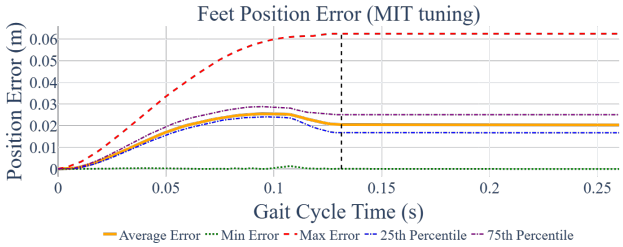


(a) Positional tracking error across obstacle densities, showing mean and maximum values.

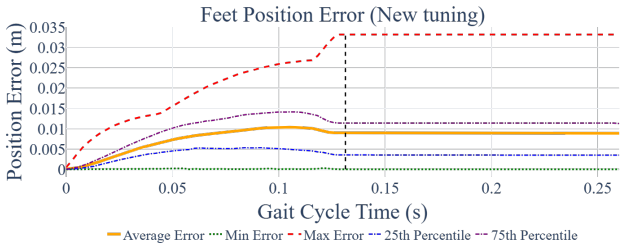


(b) Rotational tracking error across obstacle densities, showing mean and maximum values.

Fig. 12. Results of the tracking experiments. Subfigure (a) shows the positional tracking error, and subfigure (b) shows the rotational tracking error, both reported with mean and maximum values for each obstacle density.

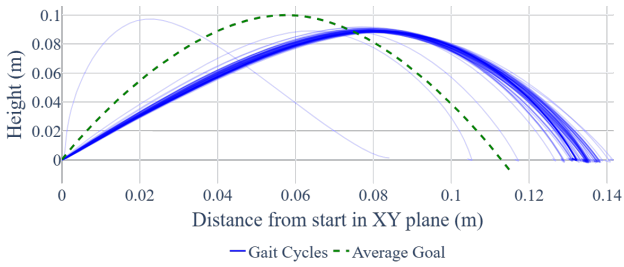


(a) Foot position tracking error with the original MIT tuning ($\mathbf{K}_{P,F} = \text{diag}(500, 500, 500)$, $\mathbf{K}_{D,F} = \text{diag}(10, 10, 10)$).

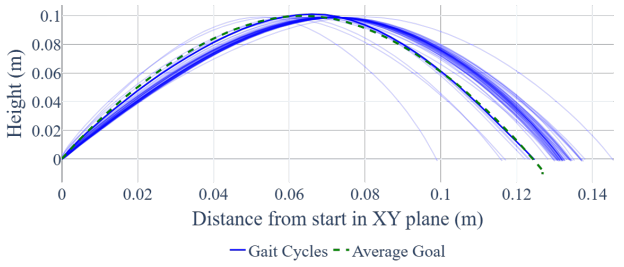


(b) Foot position tracking error with the new Bézier trajectory and tuned gains ($\mathbf{K}_{P,F} = \text{diag}(1000, 1000, 5000)$, $\mathbf{K}_{D,F} = \text{diag}(60, 60, 300)$).

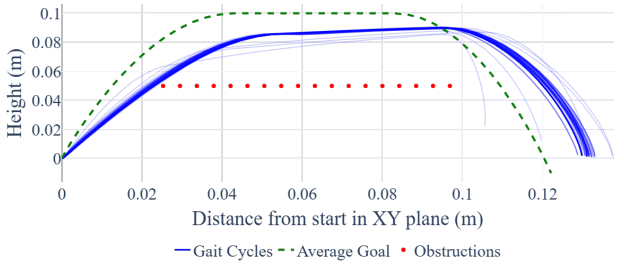
Fig. 13. Comparison of foot position tracking errors for the old MIT tuning and the new Bézier foot trajectory with tuned gains. The plots show the mean, minimum, maximum, and 25th and 75th percentile lines. The vertical dotted line marks the switch from swing to stance phase; after this line, the error represents the difference between the goal position and the final foot position on the ground.



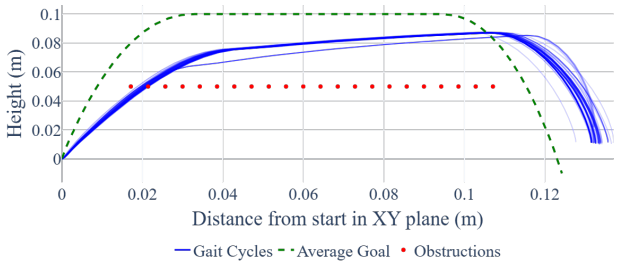
(a) Old tuning without an obstacle



(b) New tuning without an obstacle



(c) New tuning with an obstacle of 8 centimeters.



(d) New tuning with an obstacle of 9 centimeters.

Fig. 14. Comparison of the old tuning vs the new tuning of the position visualized over the entire gait cycle.